UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ

LUCRARE DE DISERTAȚIE

Evolving Virtual Creature Designs for Locomotion using Genetic Algorithms

propusă de

Alexandru Brassat

Sesiunea: iulie, 2026

Coordonator științific

Lect. Dr. Nicolae Eugen Croitoru

UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ

Evolving Virtual Creature Designs for Locomotion using Genetic Algorithms

Alexandru Brassat

Sesiunea: iulie, 2026

Coordonator științific

Lect. Dr. Nicolae Eugen Croitoru

Avizat,
Îndrumător lucrare de disertație,
Lect. Dr. Nicolae Eugen Croitoru.
Data: ..... Semnătura: .....

Declarație privind originalitatea conținutului lucrării de disertație

Subsemnatul **Brassat Alexandru** domiciliat în **România, jud. Iași, mun. Iași, bdl. Socola nr. 28, bl. Z3, et. 9, ap. 56**, născut la data de **03 aprilie 2003**, identificat prin CNP **5030403226730**, absolvent al Facultății de informatică, **Facultatea de informatică** specializarea **Inteligență Artificială și Optimizare**, promoția 2024-2026, declar pe propria răspundere cunoscând consecințele falsului în declarații în sensul art. 326 din Noul Cod Penal și dispozițiile Legii Educației Naționale nr. 1/2011 art. 143 al. 4 și 5 referitoare la plagiat, că lucrarea de disertație cu titlul **Evolving Virtual Creature Designs for Locomotion using Genetic Algorithms** elaborată sub îndrumarea domnului **Lect. Dr. Nicolae Eugen Croitoru**, pe care urmează să o susțin în fața comisiei este originală, îmi aparține și îmi asum conținutul său în întregime.

De asemenea, declar că sunt de acord ca lucrarea mea de disertație să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului ei într-o bază de date în acest scop.

Am luat la cunoștință despre faptul că este interzisă comercializarea de lucrări științifice în vederea facilitării falsificării de către cumpărător a calității de autor al unei lucrări de disertație, de diplomă sau de disertație și în acest sens, declar pe proprie răspundere că lucrarea de față nu a fost copiată ci reprezintă rodul cercetării pe care am întreprins-o.

Data: ..... Semnătura: .....

Declarație de consimțământ

Prin prezenta declar că sunt de acord ca lucrarea de disertație cu titlul **Evolving Virtual Creature Designs for Locomotion using Genetic Algorithms**, codul sursă al programelor și celelalte conținuturi (grafice, multimedia, date de test, etc.) care însoțesc această lucrare să fie utilizate în cadrul Facultății de informatică.

De asemenea, sunt de acord ca Facultatea de informatică de la Universitatea ”Alexandru-Ioan Cuza” din Iași, să utilizeze, modifice, reproducă și să distribuie în scopuri necomerciale programele-calculator, format executabil și sursă, realizate de mine în cadrul prezentei lucrări de disertație.

Absolvent **Alexandru Brassat**

Data: ..... Semnătura: .....

Contents

Abstract

The co-evolution of morphology and control in virtual creatures presents a challenging optimization problem due to the strong interdependence between body structure and behavior. This challenge is particularly evident in locomotion tasks, where effective movement emerges from the coordinated adaptation of both components. In this work, we investigate the performance of nine genetic algorithms across a variety of creature movement tasks. A large-scale empirical study comprising more than 150 parameter configurations is conducted to analyze the impact of different evolutionary strategies and parameter settings on locomotion performance.

Building upon a previous competition entry and insights gained from these experiments, we introduce `DynMut`, an enhanced evolutionary algorithm that incorporates a dynamic mutation-rate adaptation scheme, together with triggered hypermutation and a soft restart mechanism. These additions are intended to improve search diversity and mitigate stagnation while preserving useful genetic material accumulated during evolution. Experimental results demonstrate that the proposed approach is competitive with established baselines and achieves improved performance, with the soft restart mechanism providing the largest performance improvements across the evaluated tasks. The resulting algorithm was submitted to the 2026 GECCO Automated Design Competition.[1]

1 Introduction

Evolutionary robotics investigates the use of evolutionary computation techniques to automatically design and optimize autonomous agents. Rather than relying exclusively on human-designed solutions, these approaches allow both physical structures and behavioral strategies to emerge through iterative processes inspired by biological evolution. Advances in simulation environments and computational resources have enabled increasingly complex virtual organisms to be evolved, making evolutionary robotics an important area of research at the intersection of artificial intelligence, optimization, and artificial life.

A central challenge within this field is the co-evolution of morphology and control, where an agent's physical structure and behavioral policy must be optimized simultaneously. Because these components are strongly interdependent, improvements in one often depend on adaptations in the other, resulting in highly complex search spaces that can be difficult to explore effectively[21]. Consequently, the development of robust evolutionary algorithms capable of balancing exploration and exploitation remains an active area of research.

This thesis investigates the evolution of virtual creatures for locomotion tasks using genetic and evolutionary algorithms. The work focuses on evaluating different search strategies within a common simulation framework, analyzing mechanisms that influence optimization performance, and exploring an adaptive evolutionary approach designed to improve search robustness.

[1] The code of this paper is available at `https://github.com/KebabRonin/Disertatie`

The remainder of this thesis is organized as follows. The remainder of chapter 1 introduces the problem domain, presents the experimental platform and competition setting used throughout the study, and outlines the primary objectives and contributions of the thesis. Chapter 2 reviews relevant literature in evolutionary robotics, morphology-control co-evolution, diversity-preserving evolutionary search, and related competition submissions. Chapter 3 describes the experimental methodology, evaluated algorithms, and implementation details. Chapter 4 presents and discusses the experimental results. Finally, Chapter 5 summarizes the main findings, discusses limitations, and outlines directions for future work.

1.1 Problem Motivation and Background

The co-evolution of morphology and control is a fundamental problem in evolutionary robotics, concerned with the simultaneous optimization of an agent's physical structure and its behavioral policy. Small changes in morphology can invalidate or significantly alter the effectiveness of a control policy, while improvements in control often depend on specific morphological innovations. This tight coupling creates noisy and complex fitness landscapes, complicating the design of efficient optimization algorithms and motivating continued research into co-evolutionary methods.

Evolutionary approaches have demonstrated the ability to discover unconventional and highly efficient designs, with applications ranging from behavior and locomotion [31, 23] to complex hunting behaviors [6], where hand-designed solutions may be suboptimal or difficult to obtain. By exploring large design spaces without requiring explicit models of the desired behavior, evolutionary algorithms can uncover solutions that would be difficult to engineer manually.

Despite these successes, the joint search over morphology and control remains highly challenging due to the exponential growth of the design space. As the complexity of both the body plan and the control architecture increases, the number of possible interactions between components grows, making naive search methods inefficient. In addition, the presence of competing conventions and redundant representations can further hinder the search process [29].

Locomotion tasks provide a particularly demanding benchmark for studying these challenges, with applications in real-life robotics [31, 23]. Effective movement emerges from a complex interaction between morphology, sensing, control, and environmental dynamics. A seemingly minor structural modification may require a substantial change in control strategy, while improvements in control may only become beneficial when paired with suitable morphological adaptations. As a result, locomotion optimization serves as a useful testbed for evaluating evolutionary algorithms capable of coordinating changes across both body and controller.

To address these difficulties, a wide range of evolutionary strategies have been proposed, including structured representations [29, 20], adaptive evolutionary operators [3], and diversity-preserving mechanisms [22, 12]. These methods aim to improve scalability by introducing inductive biases that guide exploration

toward more promising regions of the search space. In particular, approaches that maintain behavioral diversity or exploit modularity have shown increased robustness against premature convergence[22].

Performance is often highly sensitive to representation choices, mutation operators, and parameter settings. Methods that perform well in one task may fail to generalize to others. This motivates continued investigation into adaptive and self-tuning evolutionary frameworks capable of robust performance across a variety of morphological and behavioral regimes.

1.2 Research Objectives and Contributions

This thesis investigates the optimization of virtual creature locomotion through the simultaneous evolution of morphology and control. The work is conducted within the context of the GECCO 2026 Automated Design Competition[18], which provides a standardized evaluation framework based on the Framsticks simulator[17].

The primary objectives of this work are:

- to evaluate the effectiveness of several evolutionary optimization strategies under a common experimental framework;
- to investigate mechanisms that improve exploration and reduce premature convergence;
- to analyze the impact of representation choices and evolutionary operators on search performance;
- to develop and evaluate new evolutionary algorithms across diverse optimization scenarios.

To achieve these objectives, this thesis makes the following contributions:

1. A comparative evaluation of multiple evolutionary algorithms and search strategies within a common locomotion optimization benchmark.
2. An empirical analysis of mechanisms intended to improve exploration, including adaptive mutation, hypermutation, restart strategies, and diversity-preserving techniques.
3. The design and evaluation of `DynMut`, a modified evolutionary algorithm that combines dynamic mutation-rate adaptation with triggered hypermutation and a soft restart mechanism.
4. An implementation and submission of the resulting approach to the GECCO 2026 Automated Design Competition.

The remainder of this chapter introduces the experimental platform and competition setting used throughout the thesis.

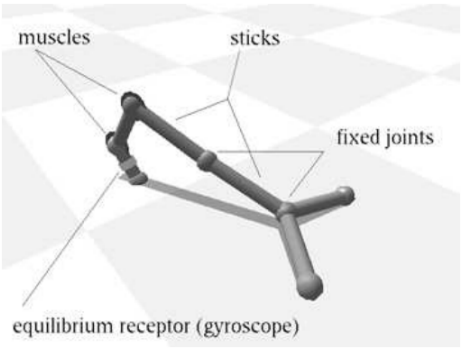


Figure 1: Components of a framsticks creature [Image Source: [19]]

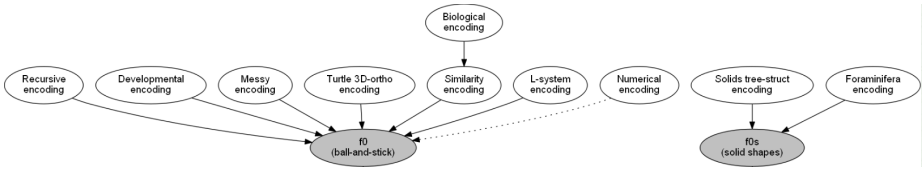


Figure 2: Genotype encodings supported by Framsticks [Image Source: [17]]

1.3 The Framsticks Simulator

The simulator used in this work is Framsticks [17], an evolutionary robotics platform that provides a physically simulated environment, built-in evolutionary operators, and support for multiple genotype representations [20]. It enables the evolution and evaluation of virtual creatures whose morphology and control systems are encoded genetically and expressed as physically simulated agents.

As shown in Figure 1, Framsticks creatures are composed of elastic body segments ("sticks"), joints, neurons (including sensors and muscles), and neural connections. Together, these components define both the physical structure of a creature and the controller that governs its behavior. Their arrangement, connectivity, and associated parameters determine how a creature interacts with its environment and ultimately influence its performance during evaluation.

The configuration of these structural and control components is specified by a genotype. A genotype encodes information such as body-part arrangement, sensor placement, neural architecture, and associated parameters. Physical properties including segment length, stiffness, orientation, and joint characteristics can be modified, as can neural parameters such as connection weights, neuron dynamics, muscle strength, and movement ranges.

The distinction between genotype and phenotype is an important one to make. The genotype is the encoded representation manipulated by evolutionary operators such as mutation and crossover, while the phenotype is the physical creature constructed from that representation and evaluated within the simulator. Evolutionary operators act on the genotype by inserting, removing, modifying,

or reconnecting encoded elements. The resulting genotype is then interpreted by Framsticks to generate the corresponding phenotype. Because modifications to the genotype can simultaneously affect both morphology and control, even small genetic changes may have significant consequences for behavior and fitness.

Framsticks supports multiple genotype representations, each providing a different way of encoding the same underlying information and introducing distinct biases into the evolutionary search process. The available representations include direct (`f0`), recurrent (`f1`), and developmental (`f4`) encodings, among others. A complete overview of the supported genotype representations is shown in Figure 2.

The `f0` genotype representation is the most expressive, as it serves as a low-level serialization of the phenotype. It can specify all attributes of individual body parts, joints, and neurons. In addition, it supports the `f0_nodel_tag` and `f0_nomod_tag` options, which can protect selected components from deletion or modification during evolution.

The `f1` genotype representation provides a more compact and human-readable description of a phenotype. Although it is less expressive than `f0` (cyclic structures cannot be represented and certain component attributes are inaccessible), it has been shown to introduce a beneficial bias in the search process for tasks such as height maximization with neural controllers disabled [20].

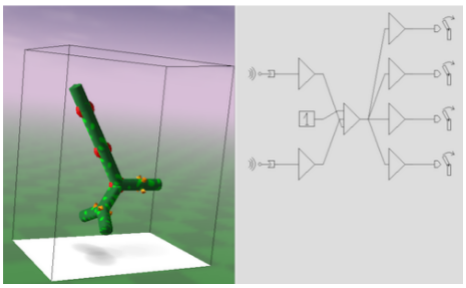


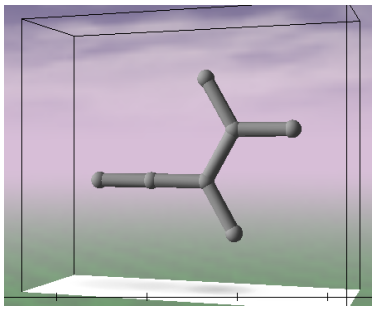
Figure 3: Framsticks genotype viewer, showcasing the body and neural structure of a creature[17]

Regardless of the genotype representation used, each genotype is ultimately translated into a corresponding phenotype and evaluated within the experiment environment (See Figure 4). The resulting creature is subjected to simulated physical forces, and its performance is measured according to the objectives of the evolutionary task. Framsticks also provides a graphical interface that facilitates experiment monitoring, visualization of evolved creatures, and inspection of both phenotype structures and their underlying genotypes.

During the evolutionary process, certain genotypes may be classified as infeasible and excluded from fitness evaluation. This may occur when a genotype contains structural warnings, violates validity constraints, produces a phenotype that fails to stabilize within the allotted simulation time, or exceeds predefined limits on the number of body parts, neurons, neural connections, or joints.

```
//0
p:
p:1.0
p:2.0
p:2.499997879272546, -0.00017002423111416295, -0.8660266114933869
p:2.499997879272546, 0.00017002423111416295, 0.8660266114933869
p:3.499997879272546, 0.00017002423111416295, 0.8660266114933869
p:1.9999936378266323, 0.00034004774107821627, 1.7320495497739516
j:0, 1, dx=1.0, 0.0, 0.0
j:1, 2, rx=1.5706, dx=1.0, 0.0, 0.0
j:2, 3, rz=-1.0472, dx=1.0, 0.0, 0.0
j:2, 4, rz=1.0472, dx=1.0, 0.0, 0.0
j:4, 5, rz=-1.0472, dx=1.0, 0.0, 0.0
j:4, 6, rz=1.0472, dx=1.0, 0.0, 0.0
```

(a) The `f0` genetic representation of the phenotype



(b) The resulting phenotype

Figure 4: Different representations of the **XRRX(X,X(X,X))** genotype in `f1` genetic representation

Among these conditions, warning-generating genotypes represent the most common source of infeasibility. In practice, evolutionary variation operators frequently create configurations in which multiple instances of the same sensor become attached to a single body part. Although such genotypes can often be parsed and converted into phenotypes, Framsticks flags them as problematic and classifies them as infeasible, preventing their participation in the evolutionary process.

1.4 GECCO 2026 Automated Design Competition

The GECCO Automated Design Competition[18] uses Framsticks[17] to provide a standardized benchmark for studying morphology-control evolution in a simulated environment, that captures both physical realism and representational flexibility. The competition emphasizes performance, robustness to different experiment environments and efficiency under constrained evaluation budgets.

This paper contributes to this setting by exploring several design choices and evaluating their impact within the competition framework. The competition setup is described on the official competition page[18], and is documented below.

The goal of the competition is to propose an algorithm that will discover agents whose center of gravity moves in the desired way (e.g. following a specific path in 3D, swinging, or jumping) in different environments used during optimization.

Algorithms will be run in 10 different evaluation environment settings (world configurations and desired movements), with each run being repeated 20 times. For each run, the best individual that was found is returned, and the final score of an algorithm in a given setting consists of the average of those best individuals.

Several simulation parameters may vary between evaluation environment settings, including:

- lifespan of a creature (”starting energy” and ”idle metabolism”)

- world size and heightfield (map)
- water level
- gravity
- creature stabilization period settings
- creature body constraints (minimal and maximal joint length)
- complexity (Genotype length, Neuron count, Joint count, etc.)

The computational constraints are:

- **Max time:** 1h (excluding time spent evaluating genotypes in the simulator)
- **Memory:** 2GB
- **Max fitness evaluations:** 100.000
- The submitted algorithm should be single-process, single-threaded, no GPU

These constraints create a realistic optimization setting in which both search efficiency and solution quality are important. Consequently, the competition serves as an appropriate benchmark for evaluating evolutionary algorithms designed to balance exploration and exploitation under limited computational resources.

2 Related Work

The co-evolution of morphology and control is a well-known but still open problem in embodied AI. The main difficulty comes from the strong coupling between an agent's body and its behavior. This makes the search space non-linear and often misleading: improving a controller can depend on changes in morphology, and even small morphological changes can break an otherwise good controller. Because of this, standard optimization methods often struggle, and approaches that support diversity and preserve useful structure have become more important.

Several main research directions have developed to address these issues. Neuroevolution methods focus on evolving neural controllers and their structure over time[29, 28]. Linkage-learning methods try to detect dependencies between parameters so that useful combinations can be exploited during the search[5, 15]. On the other hand, Quality-Diversity (QD) methods aim to produce a wide set of diverse, high-performing solutions rather than a single best one[22, 12, 31]. More recently, hybrid approaches combining evolutionary algorithms with reinforcement learning have been explored to combine global exploration with more efficient local learning[26].

Within this broader landscape, prior work in the Framsticks environment has provided useful empirical insights, particularly in competition-style benchmark setups. Studies using this platform have examined how different algorithms interact with evolving genotypes across a range of experimental conditions. Overall, this work highlights both the potential and the limitations of co-evolutionary search in complex embodied systems.

2.1 Previous Competition Submissions

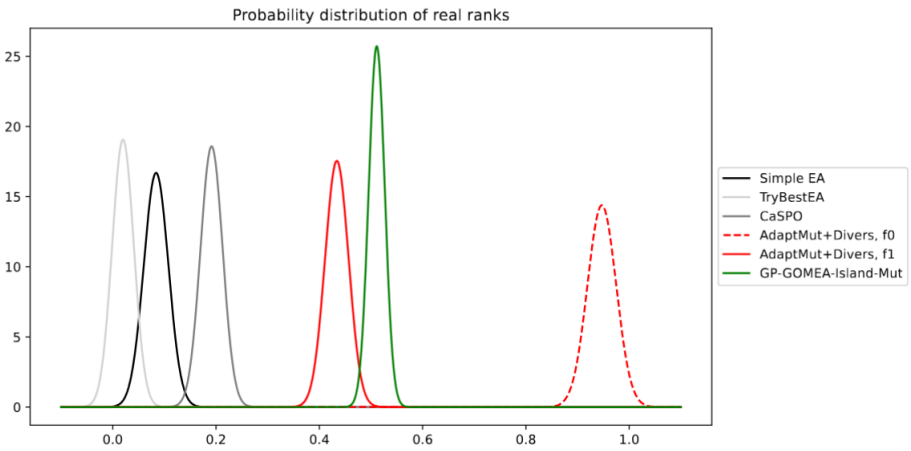


Figure 5: 2025 Competition Results [Image Source: [18]]

The following algorithms use the `f1` genetic encoding. They were submitted to the *GECCO Automated Design Competition* in past years:

Simple EA (Organizer Baseline) [18] is a conventional generational Genetic Algorithm provided by the competition organizers as a reference baseline. Individuals are represented using the native *Framsticks* `f1` genotype encoding, while variation is performed exclusively through the simulator's built-in mutation and crossover operators. Selection and replacement follow the standard genetic algorithm paradigm, making the baseline a straightforward application of evolutionary search without any additional mechanisms.

AdaptMut+Diversity (Baseline for 2026) [32] enhances the SimpleEA by introducing two novel mechanisms:

- *Adaptive Mutation Strength* - The amount of mutations applied in a generation varies depending on algorithm performance. If no significant improvement ($> 1\%$) was made in the maximal fitness from the last 4 generations, the number of mutations applied to each individual in the generation is increased (up to a maximum of 5 mutations per individual). The motivation was to help the algorithm escape the local optima.

- *Introducing random individuals* - Each mutation operation has a small probability (1%) of introducing a randomly generated individual to the population instead of mutating the current one. This allows the search space to be explored more efficiently, by introducing new genetic material.

GP-GOMEA-Island-Mut (Best entry using `f1` representation) [18] is an island-based submission, using Genetic Programming by Gene-Pool Optimal Mixing (GP-GOMEA[30]) as its central evolution engine. The initial population is randomly generated.

For each island, GP-GOMEA first converts `f1` genotypes into Genetic Programming trees (serialized in normal Polish notation, NPN). The linkage tree utilized by the GOM operator is then built by using hierarchical clustering. Thus, GP-GOMEA is able to identify closely related genes, which may benefit from being mutated together at the same time. During the application of the GOM operator, variable-length genotypes are aligned by temporary padding, allowing linkage-aware exchange of genetic material between individuals.

The algorithm is run in 10 islands $\times$ 30 individuals, with mechanisms to prevent stagnation and genetic convergence between islands. To maintain exploration, populations that fail to improve for two consecutive generations undergo mutation with probability 0.8 using the native `f1` mutation operator. Diversity between islands is monitored using the Jaccard similarity of unique genotypes; when two islands become too similar (> 0.8), one is randomly reinitialized. Furthermore, islands with low uniqueness (< 0.2) receive migrants from other islands, replacing (75%) of their individuals while retaining the remaining (25%), thereby reducing premature convergence and promoting the discovery of new solutions.

By applying the GP-GOMEA[30], a state-of-the-art algorithm, and by utilizing several mechanisms in order to ensure genetic diversity, this competition entry was able to achieve the best performance out of all entries which use the `f1` genetic encoding.

Cascaded Structure and Parameter Optimization (CaSPO) [27] is a multi-stage framework, which leverages a Large Language Model (LLM) trained on a database of successful genotypes to seed the initial population. The population is initialized with a set of individuals which was generated offline by the LLM, and is fixed for all experimental runs. The main algorithm then proceeds by iterating through the following steps in every generation:

- *Exploration* (alter Body + Neurons using an Evolutionary Algorithm (EA))
- *Disturb Control Scheme* (alter Neurons using an EA)
- *Fine-Tune Parameters* (alter Neuron Weights using an EA)

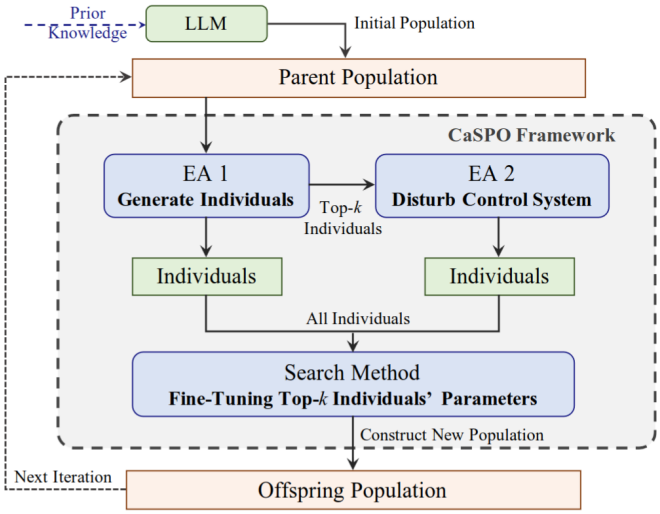


Figure 6: Illustration of the CaSPO framework [Image Source: [27]]

TryBestEA [18] performs 4 independent runs with different algorithms using the DEAP python library[7], each algorithm having an equal evaluation budget (25.000 evaluations each). The algorithms which are run are:

- *eaSimple* - DEAP[7] default library implementation; At every generation, each genotype has the chance to be potentially mutated and crossed over, with the offspring directly replacing the parents in the population.
- *eaMuPlusLambda* - DEAP[7] default library implementation; Unlike *eaSimple* above, the offspring can be generated by either mutation or crossover. The offspring are stored in a separate population from the parents, with selection being applied over both in order to determine the next generation.
- *eaMuCommaLambda* - DEAP[7] default library implementation; This method differs from *eaMuPlusLambda* through the fact that the new generation is selected from only the offspring population.
- *Custom strategy* - adjust mutation and crossover probabilities based on diversity and relative position of average fitness to median. The submission describes no further details of this strategy.

This competition entry was the only one to perform below the organizer baseline, which is most likely due to the constrained evaluation budget of each of the 4 algorithms.

2.2 Evolutionary Optimization in Framsticks

This section presents past work done in the Framsticks[17] simulator.

Early work demonstrated that the topology imposed by the encoding significantly affects search efficiency. Specifically, higher correlation between genotype and phenotype spaces leads to improved optimization performance [20]. Developmental(`f4`) and recurrent(`f1`) encodings have been proposed as more restrictive genotype encodings, with the aim of directing and reducing the search space. They were shown to be more performant than the default `f0` genetic encoding on tasks concerning maximizing height (with or without active neural networks), as well as average velocity maximization [20]. This is in stark contrast with the competition results (see figure 5), for which the only submission which uses `f0` encoding performs noticeably better than any other algorithm (even its `f1` counterpart).

Morphological evolution in Framsticks is tightly coupled with neural control. The implemented genetic operators (mutation and crossover) operate directly on structural components, such as body parts and neural connections. However, crossover has been observed to be less effective in scenarios with strong coupling between morphology and control, suggesting mutation-driven evolution may sometimes be preferable [16].

Compatible Substitutions Optimization (CoSO) [15] is an adaptation of Gene-pool Optimal Mixing Evolutionary Algorithms (GOMEA)[5] for use on gray-box problems, in particular utilizing the variable-length genotype encodings in the Framsticks environment. The algorithm replaces the Family Of Subsets (FOS) used in GOMEA with a Substitution Compatibility Function (SCF). The SCF takes as input a recipient and a donor sub-solution (a sub-solution is a subsequence of a genotype), and aims to determine the epistatic link of the two sub-solutions. That is, a low SCF implies that a substitution would not have a big impact on the resulting phenotype or its fitness.

In order to reduce the complexity of the problem space, this paper uses a very simple genotype encoding (`f9`). The encoding is inspired by turtle graphics, and consists of a string of directions: L (left), R (right), B (back), F (forth), D (down), U (up).

Once the algorithm reaches a dead end (i.e. no valid substitutions exist), an Elite Genotype Archive is constructed by trimming the population down to the genotypes with the highest SCF values, and then the algorithm is restarted with a new, randomly generated population. The Elite Archive does not take part in the genetic operations, but is maintained and provides donor sub-solutions to the GOM operator.

Several future research directions are mentioned in the paper, such as training SCF functions on past data, or maintaining an elite sub-solutions archive instead of the elite solutions archive.

Convection Selection [13] specifies a structured island population model, which can be used with any evolutionary algorithm. The core of this method consists of partitioning individuals into subpopulations based on fitness ranges (e.g. for a population of 100 individuals and 10 islands, the top 10 fitness

individuals will belong to the first island, the next 10 will belong to the second island, etc.). This method enables simultaneous exploration and exploitation by periodically merging and redistributing individuals.

Empirical results highlight the importance of diversity, particularly in early stages of evolution. Further studies[14, 2] delved deeper into the population dynamics and more problem domains, revealing that final solutions inherit genetic material predominantly from the best island in the initial population. Second chances (i.e. allowing poor solutions time to mature) showed widely varying degrees of involvement in the evolutionary process, depending on the task and problem parameters.

2.3 Neuroevolution and Incremental Complexification

Neuroevolution refers to the use of evolutionary algorithms for optimizing neural network structures and parameters. Early approaches relied on fixed-topology networks whose weights were evolved directly. While such methods can produce effective controllers, they require the network architecture to be chosen beforehand, limiting their adaptability and often resulting in unnecessarily large search spaces.

The NeuroEvolution of Augmenting Topologies (NEAT) algorithm, introduced by Stanley and Miikkulainen [29], represented a major breakthrough in the field. NEAT proposed three key mechanisms that allowed both topology and connection weights to evolve simultaneously while avoiding many of the difficulties associated with variable-length representations.

First, NEAT introduced historical markings, or innovation numbers, which assign unique identifiers to newly created genes. These identifiers provide a mechanism for aligning homologous structures during crossover and eliminate the competing conventions problem. Without such a mechanism, different network encodings representing equivalent structures could be recombined incorrectly, destroying useful functionality.

Second, NEAT employs speciation. Individuals are partitioned into species according to their structural similarity, and competition occurs primarily within species. This protects newly discovered structures from being immediately outcompeted by more mature and better-optimized solutions. As a result, innovation can survive long enough to be refined by subsequent generations.

Third, NEAT uses incremental complexification. Instead of starting from large and complicated structures, evolution begins with minimal networks and gradually increases complexity through mutation operators that add nodes and connections. This approach keeps the dimensionality of the search space manageable while enabling increasingly sophisticated behaviors to emerge over time.

Incremental complexification was studied in greater detail in subsequent work [28], demonstrating that the introduction of redundancy through mechanisms such as gene duplication allows evolution to explore alternative functionalities without sacrificing existing behaviors. Duplicated structures may initially

perform identical roles, but mutations can later specialize them into distinct modules.

The notion of complexification is particularly relevant in embodied evolution. Morphological structures and control systems are strongly coupled, and maintaining a pathway toward increasing complexity enables the emergence of specialized limbs, sensors, and control mechanisms. Rather than attempting to discover sophisticated solutions immediately, evolution progressively expands the design space, thus also encouraging simplicity in discovered solutions.

2.4 Building Blocks and Linkage Learning

Traditional evolutionary algorithms often assume that genes are independent. Mutation and crossover operate on individual variables without considering interactions among them. However, many optimization problems exhibit strong dependencies between variables, meaning that certain combinations of genes constitute building blocks that should be preserved.

Linkage-learning methods attempt to discover these dependencies automatically. One relatively recent approach is the **Gene-pool Optimal Mixing Evolutionary Algorithm** (GOMEA) [5]. Rather than relying on conventional crossover, GOMEA constructs a Family of Subsets (FOS) that captures statistical relationships among variables. These subsets represent groups of genes that are likely to contribute jointly to fitness.

The Gene-pool Optimal Mixing (GOM) operator then selectively replaces subsets of variables using information obtained from donor individuals. Candidate modifications are accepted only if they improve fitness or maintain solution quality. This greedy acceptance criterion allows GOMEA to preserve useful structures while introducing new combinations.

Compared to conventional genetic algorithms, GOMEA has demonstrated strong performance on a wide range of combinatorial and continuous optimization problems. By operating on meaningful groups of variables rather than isolated genes, the algorithm effectively manipulates building blocks and reduces destructive recombination.

In the context of morphology evolution, linkage learning is particularly appealing because many genes interact to define coherent structures. A limb, for example, may be represented by multiple parameters describing dimensions, attachment points, and articulation properties. Mutating these parameters independently may destroy functionality, whereas treating them as a unit preserves structural integrity. The Framsticks `f1` genotype encoding attempts to approximate this behavior, by introducing gene modifiers that affect multiple adjacent genes, with diminishing strength.

Linkage-learning methods can adapt automatically to different representations. As evolutionary encodings become more complex, manually identifying dependencies becomes increasingly difficult. Algorithms such as GOMEA provide a mechanism for discovering these relationships directly from population statistics.

2.5 Evolutionary Strategies

Evolutionary Strategies (ES) are a class of Evolutionary Algorithms that adapt the search parameters throughout the optimization run. Unlike traditional Genetic Algorithms, which typically rely on fixed mutation and crossover probabilities, ES algorithms dynamically adjust their search parameters based on feedback from previous generations. This self-adaptation allows the search to balance exploration and exploitation as optimization progresses, reducing the need for extensive manual parameter tuning.

Early Evolutionary Strategies introduced the concept of strategy parameters, where mutation strengths are encoded alongside the candidate solutions and evolve together with them. As successful individuals are selected, their associated strategy parameters are also propagated, allowing the algorithm to automatically adjust its search behavior to the characteristics of the optimization landscape. This principle has influenced many subsequent adaptive evolutionary algorithms.

One influential ES algorithm is Covariance Matrix Adaptation (CMA-ES)[8]. Rather than sampling mutations independently for each decision variable, CMA-ES maintains a multivariate normal distribution from which new candidate solutions are generated. During optimization, both the mean and covariance matrix of this distribution are updated using information from previously successful solutions. The covariance matrix captures correlations between decision variables, allowing the algorithm to identify promising search directions and efficiently navigate non-separable optimization problems.

CMA-ES has consistently demonstrated strong performance on continuous black-box optimization benchmarks and is widely regarded as one of the state-of-the-art methods for derivative-free optimization[24, 25]. Its ability to adapt both the overall mutation step size and the covariance structure of the search distribution makes it particularly effective on ill-conditioned and high-dimensional optimization problems. However, maintaining and updating the covariance matrix incurs a computational cost that scales quadratically with the number of optimization variables, which can limit its applicability to very high-dimensional search spaces.

2.6 Maintaining Diversity in Evolutionary Search

Premature convergence remains one of the principal challenges in evolutionary computation. Populations often become dominated by locally optimal solutions, leading to reduced exploration and stagnation. Numerous mechanisms have been proposed to address this issue.

Niching methods partition the population into subgroups that specialize in different regions of the search space. Fitness sharing mechanisms penalize individuals that are too similar to one another, thereby encouraging the maintenance of diverse solutions. Crowding techniques replace individuals with genetically similar offspring, preserving population structure across generations[12].

Novelty search[4] represents a more radical departure from objective-driven optimization. Instead of rewarding individuals according to fitness, novelty

search evaluates behavioral uniqueness. Individuals exhibiting previously unseen behaviors receive higher scores, regardless of their objective performance. This encourages exploration and helps avoid deceptive local optima.

Multi-objective optimization algorithms such as NSGA-II further incorporate diversity mechanisms. NSGA-II maintains a set of Pareto-optimal solutions while employing crowding distance metrics to preserve diversity along the Pareto front. Variants incorporating behavioral dissimilarity measures have demonstrated improved exploratory capabilities [12].

Quality-Diversity (QD) Algorithms , in contrast, aim to illuminate the problem search space as opposed to converging to a single point. QD methods simultaneously optimize for performance and diversity. Instead of maintaining one elite individual, they preserve a collection of high-quality solutions spanning different behavioral niches. This approach increases robustness and provides designers with a richer set of alternatives.

Several QD algorithms have been proposed. CMA-ME combines Covariance Matrix Adaptation with MAP-Elites to improve local search capabilities. Differential Evolution variants and NSGA-based approaches have also been adapted to emphasize diversity. Crowding and niching mechanisms can be integrated into these frameworks to maintain behavioral variation.

The appeal of QD methods lies in their ability to generate repertoires rather than single solutions. In practical applications, multiple high-performing designs may be preferable because they provide redundancy, adaptability, and insights into the underlying problem structure.

MAP-Elites Among QD algorithms, MAP-Elites [22] has emerged as one of the most influential. Instead of maintaining a single population, MAP-Elites organizes solutions into a discretized archive defined by behavioral descriptors. Each cell corresponds to a particular niche, and only the highest-performing individual within that niche is retained.

This archive-based representation transforms optimization into an illumination process. The objective is not merely to maximize performance but to populate the archive with diverse elites. Consequently, MAP-Elites provides a global overview of the search space and reveals trade-offs among behaviors.

A critical aspect of MAP-Elites is the choice of descriptors. These descriptors determine how solutions are categorized and therefore strongly influence the resulting repertoire. Common descriptors include locomotion speed, symmetry, energy consumption, and morphological characteristics.

The method has demonstrated remarkable success across a variety of domains, including robotics, soft-body design, and procedural content generation. Because the archive preserves many alternative solutions, MAP-Elites naturally supports adaptation and transfer to new environments.

However, manually defining descriptors remains challenging. Poor descriptor choices may fail to capture meaningful distinctions between behaviors. This

limitation has motivated research into automatic feature extraction methods capable of constructing descriptor spaces directly from observations.

Double Map MAP-Elites A study by [31] extends MAP-Elites by maintaining two independent archives: one for body morphologies and one for controllers. Body morphologies are evolved using Viability Evolution (ViE), which iteratively eliminates individuals that violate an adaptive viability boundary, while controllers are evolved using NEAT.

Rather than defining behavioural descriptors manually, the feature descriptors for both archives are recomputed at each generation. Principal Component Analysis (PCA) is applied to the robots' simulated spatial trajectories, and the resulting principal components are used as the descriptors that define the MAP-Elites feature spaces.

2.7 Hybrid Evolutionary and Reinforcement Learning Approaches

Recent years have witnessed growing interest in combining evolutionary algorithms with reinforcement learning. Evolutionary methods excel at exploring large and deceptive search spaces, whereas reinforcement learning provides efficient local optimization for controllers.

In many co-design systems, evolution determines morphology while reinforcement learning optimizes the corresponding control policy. This division of responsibilities allows each technique to exploit its strengths. Evolution searches globally over structural configurations, while reinforcement learning refines behaviors within each morphology.

Such hybrid approaches have demonstrated strong performance in locomotion tasks and real-life robotics[6, 9]. They often achieve faster convergence and superior solutions compared to purely evolutionary methods. Moreover, reinforcement learning enables a different paradigm for evolution: instead of evolving good final solutions, hybrid approaches will search for good learner phenotypes, that is phenotypes which enhance and support the reinforcement learning algorithms that operate on top of them[21].

Despite these advantages, hybrid systems can be computationally expensive because each morphological evaluation requires a complete reinforcement learning process. Balancing exploration and training cost remains a pressing issue in this field.

3 Methodology

This work presents an empirical study of nine algorithms and their hyperparameters, comprising of more than 150 individual experiments. Beyond the baseline algorithms, the study also evaluates several evolutionary mechanisms, including soft restarts, genotype caching, and alternative population initialization strategies. Section 3.3 provides a complete overview of the evaluated mechanisms.

3.1 Experimental Environment

The 4 example functions described in table 1 were provided as reference by the competition organizers. They were used to obtain the results presented in this paper. `evalfn3` was used to extensively explore the configuration space, while the other functions were used to check the robustness of the algorithms.

The Framsticks simulator was set up using the competition reference configuration. An extra parameter was enabled (`gen_extmutinfo = 2`) in order to allow a more granular record of mutation and crossover operations applied to an individual.

The reference competition configuration counts invalid individuals toward the evaluation budget. In this implementation, genotype validity is checked before a full evaluation is performed, allowing invalid individuals to be discarded without consuming an evaluation. This optimization is permitted under the competition rules and reduces the number of evaluations by approximately 300 on average.

While not utilized in this paper, Framsticks offers the possibility of locking certain components from being affected by mutation (`f0_nodel_tag` and `f0_nomod_tag` parameters). These attributes are key in implementing more granular mutation control, for use in building-block based algorithms or GOMEA[5]-derived works.

Optuna[1] is a python library which specializes in hyper-parameter, using Bayesian optimization. This library was used to find some of the hyper-parameter configurations that were tested. The configurations were run on the `evalfn3` test function.

Table 1: Fitness functions used for evaluation. Let $\mathbf{p}_t = (x_t, y_t, z_t)$ denote the center-of-gravity (COG) position at time step t, and let T be the number of samples in the trajectory.

ID	Formula	Description
3	$\|\mathbf{p}_0 - \mathbf{p}_{T-1}\|_2$	Euclidean distance between center-of-gravity (COG) at birth and death.
4	$\|\mathbf{p}_0 - \mathbf{p}_{T-1}\|_2 \cdot \frac{1}{T}\sum_{t=0}^{T-1} \max(0, z_t)$	Run far and have COG high above ground.
5	$1000 - \frac{\left\|\left(\frac{10t}{T-1}\right)_{t=0}^{T-1} - (z_t)_{t=0}^{T-1}\right\|_2}{\sqrt{T}}$	Rewards trajectories whose vertical coordinate follows a linear target profile from 0 to 10. The fitness is an offset and negated RMSE-like measure, to avoid negative fitnesses during maximization.
6	$\frac{1}{\sum_{t=0}^{T-1} \left\|0.5 + 0.1\sin\left(\frac{t}{100}\right) - z_t\right\| + \epsilon}$	Rewards trajectories whose vertical coordinate follows a sinusoidal target profile. The fitness is the inverse of the total absolute deviation from the target. ϵ was added for numerical stability.

The Framsticks simulator has a setting to introduce random noise to muscles/sensors, such that resulting individuals are more resilient to their environ-

ment. This setting was turned ON only for the experiments where `no_det` is mentioned in the parameters.

3.2 Evolutionary Algorithms Evaluated

The following algorithms were tested:

eaSimple DEAP[7] library implementation of a simple Evolutionary Algorithm(EA), which uses selection, mutation and crossover to evolve a population of individuals. Here, individuals are instantly replaced by their offspring, and offspring may be generated by applying both mutation and crossover operators.

eaMuPlusLambda DEAP[7] library implementation of a $(\mu + \lambda)$ EA, which applies selection over the joint population of parents(of size μ) and offspring (of size λ). Offspring generated by this method may be generated by applying only one operator: mutation or crossover (i.e. not both at the same time).

eaMuCommaLambda DEAP[7] library implementation of a (μ, λ) EA, which selects μ out of the λ offspring generated at each generation to continue the evolution. Offspring generated by this method may be generated by applying only one operator: mutation or crossover (i.e. not both at the same time).

eaMuPlusLambdaLambda $(1 + (\lambda, \lambda))$ EA, as described in [3]. With the population size set to 1, *lambda* offspring are generated through mutation, after which they are evaluated. Then, *lambda* new offspring are generated by crossing over the parent genotype with the best offspring obtained through mutation at the previous step. This method has high elitism, and has low sample efficiency, seeing as all offspring, save for the best one, are discarded.

A variant with adaptive λ computation is detailed in the original paper[3], and was implemented here. After every generation, if an offspring better than the parent was found, λ is reduced to $\lambda = \lambda/F$ (F being a constant defined as hyperparameter, usually set to 1.5). Otherwise, if no improvement was made, λ is increased to $\lambda = \lambda \cdot F^{\frac{1}{4}}$.

This algorithm generally struggled with leaving local optima, and the adaptive λ tended to explode to very high numbers in the later half of runs, leading to a tragedy of the commons w.r.t. exploring the search space.

AdaptMut[32] is a re-implementation of the competition baseline. The algorithm is described in more detail in Section 2.1.

NEAT_speciation speciation scheme inspired by the original NEAT paper[29]. Individuals are separated into different species, based on a dissimilarity threshold. At every generation, a random individual in each species is picked to be the 'centroid' of a species, and each remaining individual is assigned the species with the closest 'centroid'. The dissimilarity threshold for defining new species is

altered dynamically over the course of a run, and aiming to maintain a somewhat stable number of species in the population (the exact number of species is given as a hyperparameter). The size of each species is proportional to its performance when compared to other species. The dissimilarity metrics that were used are described in [10, 11], and are available by default in the python binding for Framsticks.

Convection Selection[13] is an island-based algorithm. At fixed intervals, all individuals are sorted by fitness and assigned to an island (so the top 10% individuals belong to the first island, the next 10% belong to the next island, etc.). This method can be used with any other EA, so it was applied over a simple EA (convection_eaSimple) and AdaptMut (convection_AdaptMut) for this experiment.

MAP-Elites[22] is a grid-based elitism method. The features used to construct the grid were the number of parts, joints, neurons and neuron connections. These were the simplest features to retrieve, and gave results which were similar to the AdaptMut baseline. This is a good starting result, as extracting more informative features could boost the performance of the algorithm significantly. Several selection methods were used to determine which elites to perturb:

- **random** - No selection bias towards a particular grid cell
- **quality** - Bias the search towards the best-performing grid cells
- **curiosity** - Bias the search towards the least explored grid cells
- **random_meta** - A random method is chosen from those described above

3.3 Ablation studies

In order to determine the behavior of different evolutionary mechansims and their impact on algorithm performance, several ablation studies were performed.

Dissimilarity measures - Several dissimilarity measures are implemented into the Framsticks library. In order to determine the best one to use when clustering the individuals into species for the NEAT_speciation algorithm, a run was made with each type of dissimilarity measure.

Overall, the `f0` genetic encoding performed consistently better than `f1`, which agrees with past results. Some `f1` experiments terminated due to time constraints, while no configurations tested on `f0` had such problems. This can be explained by the additional conversion step required when using `f1`, which consists of converting the genotype into the `f0` representation first, before being able to construct the phenotype.

For the `f1` genetic encoding, the dissimilarity measures with the best performance were PHENE_STRUCT_GREEDY and PHENE_STRUCT_OPTIM. The experiment's mean fitnesses were similar, although PHENE_STRUCT_GREEDY was a bit more stable. For the `f0` encoding, PHENE_STRUCT_OPTIM was the best overall configuration.

The PHENE_DENSITY family of dissimilarity measures took a very long time to compute and was terminated due to time constraints, dramatically reducing the number of evaluations performed (2k evaluations, out of the alotted 100k total evaluation budget). As a result, they are an impractical choice for usage in the GECCO competition. The reduced performance is due to having to sample a number of random points in 3D space in order to compute the dissimilarity score for each pair of individuals.

Table 2: Ablation study on dissimilarity metrics, run on `evalfn3`. The Overtime column shows how many runs were terminated due to time constraints, as opposed to exceeding the evaluation budget.

Nr.	Stddev	Mean	Median	Max	Overtime	Name
1.	73.01679	**212.67342**	**204.31950**	376.39000	(0/20)	F0_PHENESTRUCTOPTIM
2.	57.04898	189.49424	180.27850	303.23000	(0/20)	F0_PHENESTRUCTGREEDY
3.	62.84837	172.29113	157.24650	318.19800	(3/20)	F1_PHENESTRUCTGREEDY
4.	99.68314	168.83877	128.41600	**440.29700**	(4/20)	F1_PHENESTRUCTOPTIM
5.	74.73547	167.46402	155.23850	357.24000	(0/20)	F0_FITNESS
6.	76.95924	146.69567	135.20300	316.40900	(0/10)	F1_FITNESS
7.	47.04598	74.61855	66.99510	194.15300	(0/20)	F1_GENELEVENSHTEIN

Continued on next page

Table 2 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
8.	27.95409	26.19581	13.71005	86.35990	(20/20)	F1_PHENEDENSITYFREQ
9.	17.11060	13.11953	4.41728	63.76950	(20/20)	F1_PHENEDENSITYCOUNT
10.	25.35968	11.21698	3.47358	119.45800	(20/20)	F1_PHENEDESCRIPTORS

Genotype Caching - Some genotypes may have the chance to be re-evaluated multiple times during a run. A simple genotype cache was used to get the fitness of previously evaluated individuals. This can happen if the available search space of the algorithm is small, such as after seeding the population with the same genotype, or in the case of premature convergence. The cache that was implemented only replaced exact genotype matches. This saves some evaluations in the first generations (when the accessible search space is still small, and collisions are more common). This is most useful when the initial population is not random. Overall, the fitness impact is negligible.

Invalid individual repairing - When an individual is found to be invalid, it is removed from taking part in evolutionary processes by default. A different approach is to attempt to repair the genotype, in order to salvage potentially good solutions from the infeasible space of the fitness landscape. Individual repair was done by repeatedly applying mutations until the genotype becomes feasible or a maximum of 100 mutations is reached.

This method led to a performance boost of 19.55%. However, the Mann–Whitney U test did not indicate a statistically significant difference at a significance level of 0.05 (p=0.067).

Restart - Reinitialize the population after *patience* generations without improvement. This mechanism is easy to implement and brings the greatest increase fitness, since most evaluated algorithm configurations struggle with stagnation in the later half of their runs. Several methods were implemented:

- *soft_perturb_best* - take the best individual found in this run, and create a new population by mutating it 0-4 times (25% population) and adding random individuals (75%). The ratios were adapted from the migration operation in GP-GOMEA-Island-Mut[18], a previous competition entry.
- *soft_perturb_best_all* - reinitialize the population as above, but using the best individual over all previously restarted runs (not just current run). This method performed worse than *soft_perturb_best*, which is likely due to being unable to escape the pull of the same local optimum over multiple restarts.
- *hard* - reinitialize the population from scratch, discarding all generated individuals up to that point.

Overall, `soft_perturb_best` had the best performance, since it preserves knowledge gathered in previous attempts, but also allows for escaping local minima. With no restarts, algorithms struggle with stagnant populations, especially since the best solution is usually found after 60k evaluations (out of the total 100k evaluation budget). After adding soft restarts, not only are the best solutions of better quality, but they are also found later in the evolutionary process (79k evaluations), indicating a more robust evolutionary process, that avoids stagnation and makes good use of the available evaluation budget.

For the baseline `AdaptMut` algorithm, adding a `soft_perturb_best` restart with 10 generations patience yielded a fitness increase of +33.1%, making it one of the top 5 tested methods.

After repeating the experiment for the `eaSimple` algorithm with `f0` encoding, a statistically insignificant drop in performance was observed.

The results of this ablation study suggest that the substantial performance improvement observed in the `AdaptMut` configuration is unlikely to be explained by either the restart mechanism or adaptive mutation strength alone, and may instead result from their interaction. Further experiments are required to validate this hypothesis.

Population Initialization - By default, the entire population of an algorithm is initialized with copies of the simplest possible genotype, which consists of a single stick. With no muscles, joints or neurons present, this means that the first couple of generations will predominantly consist of immobile structures, left to navigate a fitness plateau. In order to skip these warm-up generations, some alternate initialization methods were employed:

- Initialize with simplest mobile genotype (XX) - Instead of initializing the population with the single-stick genotype, the initial genotype consists of two sticks, in a hinge-like structure. The motivation for this was to start the algorithm closer to the region of interest in the genotype space (i.e. mobile structures)

- Initialize with neuron-library genotype (neurons) - Extending the motivation described above, a mobile structure should contain at least one neuron and one muscle. To aid in finding mobile structures, the initial genotype was set to contain one of each neuron and muscle defined in Framsticks. This initialization method was inspired by inspecting the best genotypes found after some preliminary runs, which contained a large amount of unused neurons.

- Random initial genotypes (random) - This method has a better chance of generating a mobile structure on the first generation, and it also begins with a set of diverse solutions, aiding in the search process

Each initialization method was tested on eaSimple and AdaptMut, using both `f0` and `f1` genotype encodings. While only `f1` showed an upwards trend following the change in initial population, none of the evaluated ablations were

significantly different when compared to the baseline (Mann-Whitney U test, significance level 0.05).

Selection - The competition reference implementation uses *tournament* selection. A batch of experiments was conducted with the *roulette* selection method.

AdaptMut with soft restarts on `evalfn3` was used as a baseline for this ablation. The *tournament* selection scheme was found to have the best performance, while the other method showed a significant fitness decrease (-22.3%).

The results would indicate that a tight balance between diversity of solutions and elitism is needed for a good performance. *Tournament* selection offers a good mix of fitness-based discrimination and population diversity through random local competition.

MAP-Elites cell choice - Out of all evaluated cell choice strategies, the quality bias had the best performance by far, while the curiosity strategy was outclassed by the naive random choice strategy.

Table 3: Ablation study on MAPElites grid cell choice method, run on `evalfn3`. Ablation done on a MAPElites with `f0` genetic representation, 20 population size for crossover operation, soft_perturb_best restart with 100 generation patience, and picking the grid cells among the top 50 when scored with that particular selection method.

Nr.	Stddev	Mean	Median	Max	Overtime	Name
1.	118.59155	**326.90425**	**306.65950**	**679.97300**	(0/20)	MAPElites_quality
2.	84.10663	265.23600	241.93900	507.45900	(0/20)	MAPElites_random_meta
3.	57.18260	120.94354	108.61950	285.34300	(0/20)	MAPElites_random
4.	23.27419	81.54561	76.79185	144.88900	(0/20)	MAPElites_curiosity

Genetic encoding - The `f0` encoding consistently outperformed `f1` for all evaluation settings, as exemplified by the previous competition submissions. Our experimental results corroborated this result: eaSimple showed a $+37.4\%$ mean increase when switching from `f1` to `f0`, while NEAT_speciation showed a $+83.1\%$ increase. This is in contrast with previous work[20], suggesting that the difference in genotype encoding performance may be related to the type of tasks that are evaluated in the competition environment and in the prior work.

One reason for this result is the more expressive parameters available in `f0`: joint stiffness is inaccessible to `f1` genotypes, but it is consistently used to generate good `f0` solutions. This is especially of use in the competition task, which fully utilizes neural networks to achieve movement. Prior work[20] only contained comparisons on tasks where the neural control system was disabled.

3.4 DynMut

After doing a survey of the algorithms described above, a new approach was developed, which is described below. DynMut uses AdaptMut (the algorithm with the best performing configuration) as a basis, and implements an Evolutionary Strategy with soft restarts on top.

This algorithm takes the hypermutation introduced by AdaptMut, and builds upon it by dynamically updating mutation probabilities over the course of evolution. To this end, Framsticks offers simulation parameters that can alter the behavior of mutation. More exactly the probability of each type of mutation(add neuron, remove part, modify parameters, etc.) can be set granularly for each evaluated individuals.

The method is designed to reduce reliance on static mutation schedules by introducing feedback-driven adjustment mechanisms that operate at the level of mutation types. Unlike classical evolutionary strategies that primarily adapt continuous strategy parameters (e.g., step sizes), DynMut focuses on structuring and updating a discrete distribution over mutation operators, coupled with the single global mutation strength scalar provided by AdaptMut.

At the core of the algorithm is a standard generational evolutionary loop with selection, variation, evaluation, and replacement. Variation is implemented through a combination of mutation and crossover, where mutation is applied iteratively according to a global mutation strength parameter (m_s). This parameter directly controls the number of sequential mutation operations applied to each selected individual, thereby acting as a coarse-grained control over exploration intensity. DynMut maintains a single shared mutation strength for the entire population, which is adjusted over time based on observed improvements in population fitness.

The central adaptive component of DynMut is the global mutation statistics model (S), which aggregates feedback on mutation operator performance. After offspring evaluation, each applied mutation is credited according to its contribution to the observed fitness change (Δf), normalized by the number of mutation operators involved in producing a given individual. This equal credit assignment reflects the partial observability of operator effects when multiple mutations contribute to a single genotype transition. The resulting signal is accumulated in S, and is normalized to maintain a stable scale.

The mutation rate normalization step transforms the mutation statistics into a distribution over mutation operator types, preventing the accumulation of unbounded scores over time. The normalization also plays a secondary role, of ensuring that no mutation type goes extinct during a run. This is achieved by adding the mean mutation rate to each individual mutation rate and normalizing the resulting values by their total sum.

The mutation strength adaptation mechanism operates in parallel with the operator-level learning process and is driven by stagnation detection in the fitness trajectory. Specifically, the algorithm maintains a history of best observed fitness values and compares recent progress against a short historical window. If improvement falls below a relative threshold, the mutation strength (m_s)

is increased multiplicatively, promoting more aggressive exploration through deeper application of mutation operators. Conversely, when sufficient progress is observed, m_s is decreased to favor exploitation and refinement.

If no improvement is found in 10 generations, a more drastic evolutionary mechanism is activated: re-initializing the population using a soft restart scheme (25% top individuals from the current population, and 75% random individuals to aid in exploration). DynMut preserves variables such as m_s and S across restarts.

Overall, DynMut can be interpreted as a hybrid between a standard evolutionary algorithm and an online learning system over mutation operators. The global statistics model S provides a data-driven estimate of operator utility, while the mutation strength controller regulates the intensity of variation in response to stagnation signals. Together with soft population restarts, these mechanisms yield a self-regulating evolutionary process in which both operator selection biases and search aggressiveness are dynamically adapted based on observed fitness dynamics, without introducing per-individual strategy parameterization.

Function UpdateMutationScores

function UpdateMutationScores($S, x, M, \Delta f$)
  $k \leftarrow |M|$
  for each mutation type $m \in M$ **do**
    $mutfraction \leftarrow \frac{|m \in M|}{|M|}$
    $S[m] \leftarrow \text{scorefn}(x, M, \Delta f) \cdot mutfraction$
  end for
  Normalize
  $Z \leftarrow \sum_m S[m] + \epsilon$
  for each mutation type m **do**
    $S[m] \leftarrow (S[m] + \text{mean}(S))/Z$
  end for
  return S
end function

Algorithm DynMut

Require: Population P, initial mutation probabilities $\mathbf{p}$, mutation step sizes σ
Require: Global statistics decay factor d
Require: Fitness function $f(\cdot)$
Require: Score function scorefn$(x, M, \Delta f)$

Initialize population P
Initialize global mutation statistics $S \leftarrow \emptyset$
Initialize fitness history $H \leftarrow [\,]$
Set mutation strength $m_s \leftarrow 1$
Initial Evaluation
for each individual $x \in P$ **do**
    Evaluate fitness $F(x)$
end for
while termination condition not met **do**
    $P_{\text{sel}} \leftarrow \textsc{Select}(P)$
    Variation (mutation + crossover)
    for each individual $x \in P_{\text{sel}}$ **do**
        if rand() $< p_{\text{mut}}(x)$ **then**
            for $i = 1$ to m_s **do**
                $x \leftarrow \textsc{Mutate}(x)$
            end for
        end if
        if rand() $< p_{\text{cross}}$ **then**
            $y \leftarrow \textsc{Select}(P_{\text{sel}})$
            $(x, y) \leftarrow \textsc{Crossover}(x, y)$
        end if
    end for
    Evaluation and statistics update
    for each individual $x \in P_{\text{sel}}$ **do**
        Evaluate fitness $F(x)$
        $\Delta f \leftarrow F(x) - F(\text{parent}(x))$
        $S \leftarrow \textsc{UpdateMutationScores}(S, x, x.\text{mutations}, \Delta f)$
    end for
    Append $\max_{x \in P_{\text{sel}}} F(x)$ to H
    AdaptMut mutation strength update
    if $|H| > 4$ **then**
        if $|H_{t-4} - H_t| < 0.01 \cdot H_t$ **then**
            $m_s \leftarrow 1.1 \cdot m_s$
        else
            $m_s \leftarrow 0.9 \cdot m_s$
        end if
        $m_s \leftarrow \textsc{Clamp}(m_s, 1, 5)$
    end if
    if $|H| > 10$ and $H_{t-10} > H_t$ and $max(H_t, ..., H_{t-10}) = H_{t-10}$ **then**
        $P' \leftarrow \text{top}_{25\%}(P') \cup \text{randomIndividuals}(0.75 \cdot |P'|)$
    end if
end while

The mutation rates are computed based on global performance of each mutation type, which is computed by utilizing the number of mutations that resulted into fitness increases and decreases, as well as a rolling window over the raw values of those fitness changes. In cases where multiple operations contributed to the generation of a genotype, equal weight was given to all applied mutations (so a genotype that obtained a -5 fitness decrease, and was generated by crossover and a part add mutation, would be counted as -2.5 fitness decrease for the crossover operator, and -2.5 for the part add mutation type). The crossover rate was not altered by these statistics.

Several scoring functions were considered when computing the new probabilities. They are detailed below, along with Cohen's d test when compared to the baseline (the default, organizer-defined mutation rates, with no adaptive mutation).

- **neg** (0.198 Cohen's d test - best performance) - Give the biggest probability to the mutation type that causes the biggest fitness decrease. This scoring function performed the best out of all, result that was surprising, since this method would seemingly promote incompetence.
- **ratio_fifthrule** (-0.095 Cohen's d test) - loosely based on the 1/5-th success rule[3], this method gives the biggest probability to mutation types which have a 1/5 success rate (i.e. fitness increases / total count)
- **ratio_v2** (-0.395 Cohen's d test) - give biggest probability to mutation types which have highest fitness increases / total count ratio
- **neg_conservative** (-0.506 Cohen's d test) - Give the biggest probability to the mutation type that causes the smallest fitness decrease.
- **pos** (-0.563 Cohen's d test) - Give the biggest probability to the mutation type that causes the biggest fitness increase.
- **const** (-0.585 Cohen's d test) - Equal probabilities for all mutation types.

In order to avoid the extinction of a certain mutation type, all mutations probabilities were forced to be above 0, even when the scoring function that was used might suggest otherwise.

A distinguishing feature of the **neg** scoring function is the increased variability of the mutation rates compared to other scoring variants. In particular, the scoring scheme induces a stronger redistribution of probability mass across mutation types, which results in a more dynamic mutation selection process over time.

This behavior may periodically introduce an implicit bias toward certain mutation types, even though the scoring function appears, at first glance, to promote under-performing mutations. The resulting search dynamics differ other from more uniform mutation-rate adaptation schemes, as the mutation distribution does not converge to a single dominant operator but remains more dispersed throughout the run.

These observations suggest that the performance of the `neg` scoring function is driven not simply by rewarding low-performing mutations, but by the induced structure of the mutation probability distribution. However, the precise relationship between this redistribution mechanism and task performance remains unclear and requires further analysis.

4 Results

Following an initial hyperparameter search across multiple algorithms, AdaptMut was selected for further fine-tuning based on its performance. Optuna subsequently identified its best-performing hyperparameter configuration, which ranked among the three best-performing algorithm configurations evaluated in this study.

Based on preliminary results, AdaptMut was selected as the foundation for the development of DynMut. The resulting algorithm achieved performance comparable to the best-performing AdaptMut configuration and reached this level of performance with relatively little hyperparameter tuning.

In total, 197 experiments were conducted (3626 individual runs), with the raw log files taking up around 60 GB. The final results are available in condensed JSON format on the paper's public repository[2].

Below are the results for each tested evaluation function:

Table 4: Leader board of the best variant (w.r.t. mean) of each tested algorithm, on `evalfn3`. The baseline is marked in italic. For a list of all tested configurations, see Appendix B

Nr.	Stddev	Mean	Median	Max	Name
1.	118.56489	**361.60925**	318.37850	645.18800	DynMut
2.	128.03199	342.58060	**326.74850**	608.65300	AdaptMut
3.	118.59155	326.90425	306.65950	**679.97300**	MAPElites
4.	120.81813	289.86785	275.52600	529.32700	convection_AdaptMut
5.	60.06200	250.20824	265.23500	326.23300	NEAT_speciation
6.	102.88860	249.01476	243.49050	509.24100	*AdaptMutF0pmut08 - baseline*
7.	75.62944	245.00700	258.95100	379.95700	convection_eaSimple
8.	103.76094	188.69956	189.65600	474.34500	eaSimple
9.	65.36974	177.93512	170.24400	312.38400	eaMuPlusLambda
10.	70.11401	168.04577	156.33850	336.39300	eaOnePlusLambdaLambda

[2] `https://github.com/KebabRonin/Disertatie`

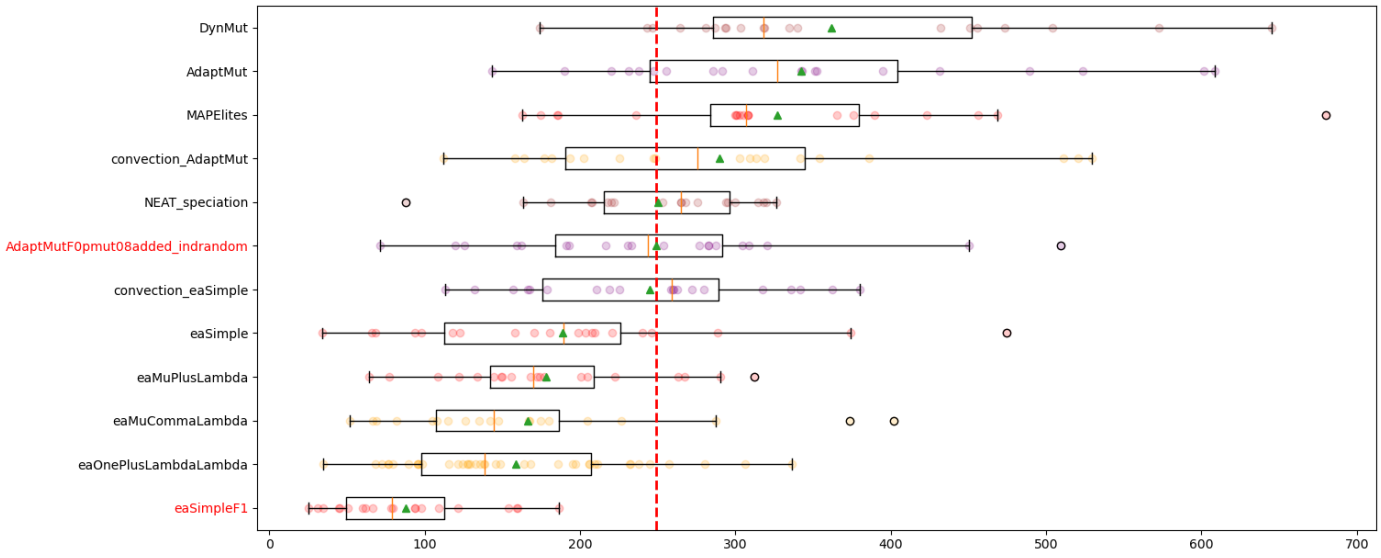


Figure 7: Plot of run distributions for `evalfn3` (see table 1) the best configuration of each algorithm. Baselines are highlighted in red. A triangle marks the mean, and a line marks the median of an algorithm. A dotted red line signifies the mean fitness of the baseline.

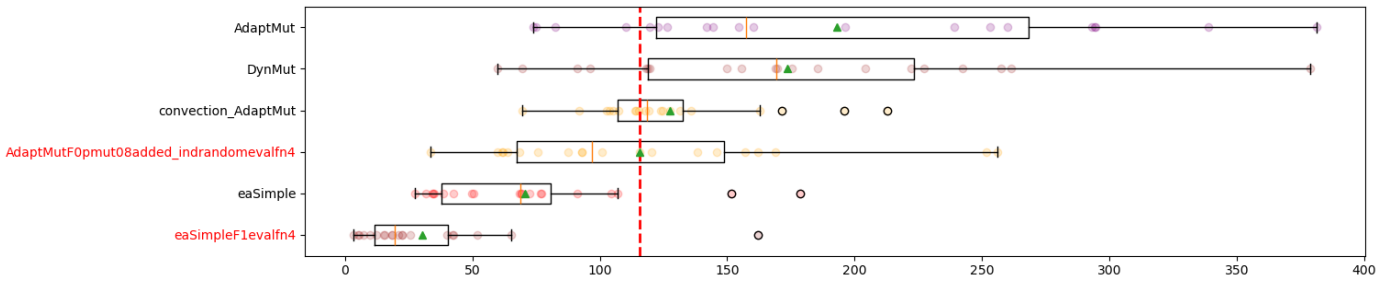


Figure 8: Plot of run distributions for `evalfn4` (see table 1) the best configuration of each algorithm.

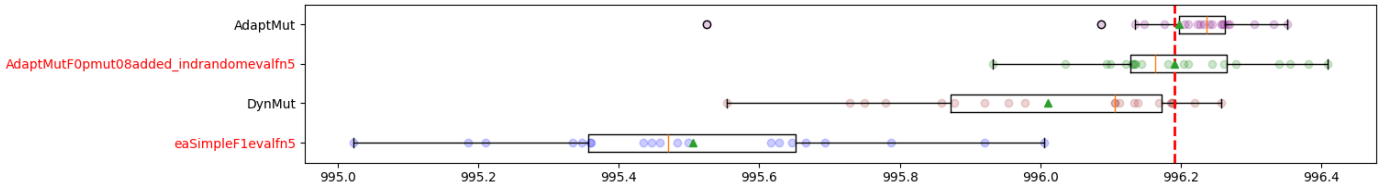


Figure 9: Plot of run distributions for `evalfn5` (see table 1) the best configuration of each algorithm.

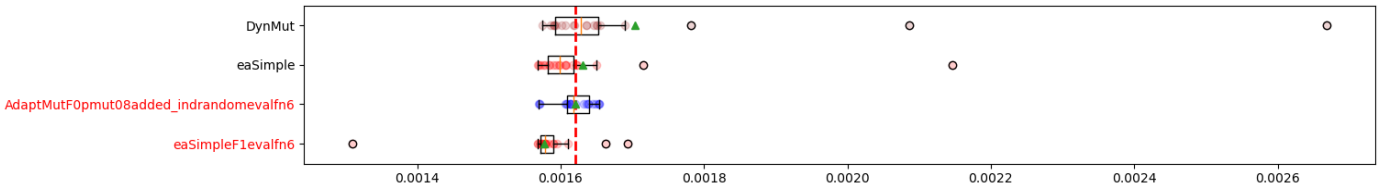


Figure 10: Plot of run distributions for `evalfn6` (see table 1) the best configuration of each algorithm.

All reported results are averaged over 20 independent runs unless stated otherwise. Performance is measured using the best individual found during a run. We report the mean, median, maximum observed fitness, and standard deviation to characterize both solution quality and search stability.

The strongest overall configuration for `evalfn3` employed the `DynMut` algorithm, using the `f0` genotype representation and a soft restart with patience 10. This configuration achieved slightly better results over the baseline for `evalfn3` and `evalfn4`. For `evalfn5` the results were below the baseline, and `evalfn6` showed no significant differences.

4.1 Genotype Insights

This chapter presents some insights and statistics gathered after analyzing the entire genotype pool produced by all experimental runs produced as part of this work. A full list of genotypes can be accessed at this paper's public repository (`https://github.com/KebabRonin/Disertatie`). The `globalhof.gen` file can be imported directly into the Framsticks GUI application, such that genotypes can be inspected in greater detail.

Genotype statistics - Out of all recorded solutions over all evaluation functions, 26.74% used the `f1` encoding. Out of the remaining `f0` solutions, 41.01% contained cycles.

When taking only the top 20% best-performing solutions on `evalfn3`, the statistics change as follows: 11.65% of solutions used the `f1` encoding, and 64.52% of `f0` solutions contained cycles.

Figure 11 presents some common genotype features, aggregated over all discovered solutions. The number of parts and joints peaks at around 5 of each, regardless of the genotype encoding which is used. This is in contrast with the neuron and neuron connection counts, which differ greatly between encodings: while `f0` strongly converges to solutions with around 12 neurons and a low number of connections, the `f1` solutions are more uniformly distributed. This may suggest that, for `f0`, neuron connection breakages are more likely to occur, justifying the low number of connections present in the final solutions.

Solutions which use `f0` have on average, unsurprisingly, double the length of `f1` genotypes. One of the strengths of `f1` is, after all, its human-readable format.

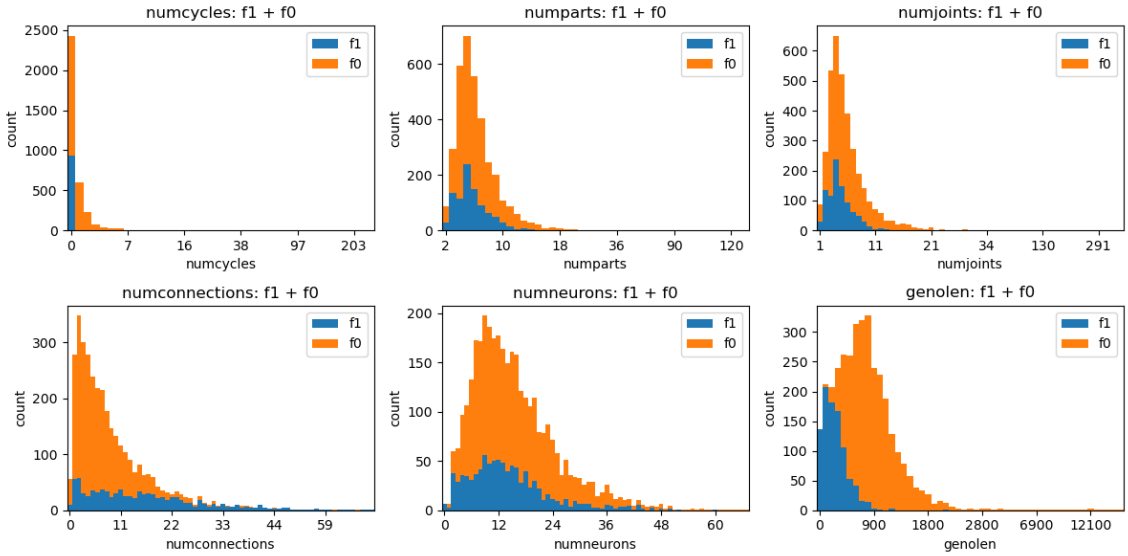


Figure 11: Statistics extracted from the best genotype from each run. No significant differences in distributions were found when filtering by the top 20% solutions.

Best `evalfn3` solution The best found solution using `f1` has a jumping behavior, and consists of a simple and straight morphology (See Figure 12). The controller contains a lot more connections than the `f0` solution, although one particular connection is present multiple times, with different weights.

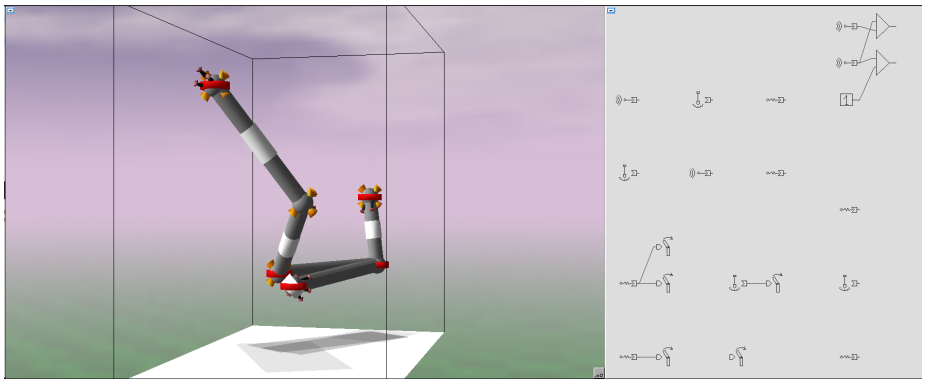
The best `f0` solution has a crawling behavior, enabled by the two prongs in a V shape, which are used for supporting the main body. The controller contains a lot of neurons, but few connections. Around half of all neurons have no function in the resulting genotype, remaSining disconnected from any muscles.

4.2 Discussion

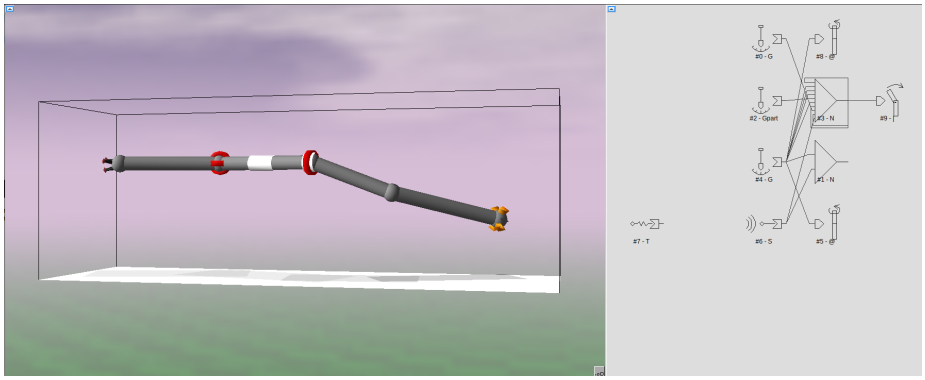
Soft restart performance The soft restart mechanism accounted for 33% of the improvement seen in the best evaluated algorithm configurations. This is by far the biggest improvement over the baseline, with little implementation complexity.

While multiple restarts schemes were tested, the best performance was *soft_perturb_best*.

DynMut negative scoring A surprising result was the performance of the best scoring function: giving the mutation types with the worst performance the highest probability.



(a) Best `f0` genotype for the `evalfn3` test function



(b) Best `f1` genotype for the `evalfn3` test function

Figure 12: Showcase of the genotypes with the best performance on `evalfn3`

This remains an open question, although one hypothesis is that the non-standard normalization scheme is too eager, implicitly reducing mutation rate variability in an effort to avoid the extinction of any mutation type.

Further study is needed to determine the merit of these findings, and to find more robust normalization schemes.

Genetic encoding performance Prior work[20] identified the `f0` genotype representation as having the weakest performance on body-only height maximization tasks. In contrast, for the competition task of evolving movement behaviors, `f0` exhibits the best performance. This can be attributed to `f0` allowing the formation of cyclic part configurations, and, most importantly, providing access to more part attributes (like stiffness, rotational stiffness, more granular x-y-z stick positioning). Proof of this claim is that the best found solution over all attempted configurations contained no cycles, and was found by an algorithm using the `f0` representation.

One interpretation for the large amount of unused neurons would be the storage of building blocks for possible future use, or, more likely, the remains of past fitness-neutral mutations or breakages caused by perturbation mechanisms.

Emergent physical communication between body segments A big part of the creature movement of the top solutions is caused by bending muscles, which require no neural connections to be effective: they always seek to orient the parts in a certain angle relative to the previous part. By utilizing several muscles, with conflicting angles, movement can be achieved with no direct connections between neurons, opting to use indirect connections, enforced by the physical structure of the creature.

4.3 Future Work

- A more thorough study of the soft restart mechanism, and its interactions with different algorithms and hyper-parameter configuration.
- A study on the performance impact of using `f0` representation wile disallowing cycles in the structure over the default `f0` representation.
- An additional study on DynMut and other mutation rate score function choices, along with alternative normalization methods.
- Employ graph analysis techniques to gain new insights into `f0` genotype-to-fitness correlations, some of which could be used to construct new algorithms or augment existing techniques. Some directions would be: finding quicker and more performant crowding/dissimilarity/linkage metrics, in order to identify promising building blocks or teacher/student genotypes.
- Use `f0_nodel_tag` and `f0_nomod_tag` options available in Framsticks, in order to prevent mutations from affecting certain high-value genotype

innovations. This may open the door to more advanced algorithms and other evolutionary mechanisms.

- Adapt partition crossover for use in crossover or determining dissimilarity. An interesting challenge would be to adapt it to the cyclic structures possible with the `f0` genetic representation.

5 Conclusion

This work investigated the performance of various evolutionary algorithm configurations on the virtual creature locomotion problem defined by the GECCO Automated Design Competition[18]. A broad survey of evolutionary methods was conducted, including classical evolutionary algorithms, speciation-based methods, MAP-Elites, and island-based approaches, together with several evolutionary mechanisms such as restart strategies, adaptive mutation, genotype caching, and alternative population initialization techniques. In total, over 150 parameter configurations spanning nine algorithms were evaluated through 197 experiments (3626 individual runs). Building on the strongest-performing baseline, a novel adaptive mutation framework, DynMut, was developed and evaluated. The best-performing DynMut configuration employed soft restarts, triggered hypermutation, and dynamic mutation probabilities, and was submitted to the 2026 GECCO Automated Design Competition.

Among all evaluated evolutionary mechanisms, soft restarts produced the largest performance improvement (approximately +33%). The experimental results further demonstrated that the `f0` genotype representation consistently outperformed `f1` on the evaluated locomotion tasks, highlighting the importance of representation choice for morphology-controller co-evolution. Although DynMut achieved the best overall performance on the primary evaluation task, its improvement over the best fine-tuned AdaptMut configuration was relatively small. Likewise, a MAP-Elites implementation using only simple morphology-based descriptors and comparatively little hyperparameter tuning achieved competitive performance, suggesting that quality-diversity methods remain a promising direction for future work. Overall, the results indicate that adaptive mutation operator selection is a viable extension of existing evolutionary approaches, although the mechanisms underlying its effectiveness require further investigation.

Generative AI Disclosure

This paper has used generative AI for the following purposes:

- Help in writing introduction, conclusion and literature review sections, proofreading;
- Writing the optuna hyperparameter search scripts;

- Various code snippets (working with plots & dataframes, latex tables & formatting);
- Using *Deep Research* to get a list of papers related on the subject. All papers which were obtained in this way were then manually reviewed for a better understanding of the problem domain and past results;
- Debugging, paper explanations;

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Sofya Aksenyuk, Szymon Bujowski, Maciej Komosinski, and Konrad Miazga. Late bloomers, first glances, second chances: Exploration of the mechanisms behind fitness diversity. In *Genetic and Evolutionary Computation Conference (GECCO '24)*. ACM, ACM, 2024.

[3] Benjamin Doerr, Carola Doerr, and Franziska Ebel. Lessons from the black-box: fast crossover-based genetic algorithms. In *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation*, GECCO '13, page 781–788, New York, NY, USA, 2013. Association for Computing Machinery.

[4] Stephane Doncieux and Alexandre Coninx. Open-ended evolution with multi-containers qd. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO '18, page 107–108, New York, NY, USA, 2018. Association for Computing Machinery.

[5] Arkadiy Dushatskiy, Marco Virgolin, Anton Bouter, Dirk Thierens, and Peter A. N. Bosman. Parameterless gene-pool optimal mixing evolutionary algorithms, 2021.

[6] Dario Floreano and Laurent Keller. Evolution of adaptive behavior in robots by means of darwinian selection. *PLoS biology*, 8:e1000292, 01 2010.

[7] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner, Marc Parizeau, and Christian Gagné. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, 13:2171–2175, jul 2012.

[8] Christian Igel, Nikolaus Hansen, and Stefan Roth. Covariance matrix adaptation for multi-objective optimization. *Evolutionary Computation*, 15(1):1–28, 2007.

[9] Tianyi Jin, Melya Boukheddimi, Rohit Kumar, Gabriele Fadini, and Frank Kirchner. Evolutionary continuous adaptive rl-powered co-design for humanoid chin-up performance, 2025.

[10] Maciej Komosinski, Grzegorz Koczyk, and Marek Kubiak. On estimating similarity of artificial and real organisms. *Theory in Biosciences*, 120(3-4):271–286, December 2001.

[11] Maciej Komosinski and Agnieszka Mensfelt. A flexible dissimilarity measure for active and passive 3d structures and its application in the fitness–distance analysis. In Paul Kaufmann and Pedro A. Castillo, editors, *Applications of Evolutionary Computation*. Springer, Springer, 2019.

[12] Maciej Komosinski and Agnieszka Mensfelt. Enhancing quality-diversity optimization through domain-specific dissimilarity as crowding distance. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '25, page 898–906, New York, NY, USA, 2025. Association for Computing Machinery.

[13] Maciej Komosinski and Konrad Miazga. Tournament-based convection selection in evolutionary algorithms. *PPAM 2017 proceedings, Lecture Notes in Computer Science*, 10778:466–475, 2018.

[14] Maciej Komosinski and Konrad Miazga. Revealing the inner dynamics of evolutionary algorithms with convection selection. In *Genetic and Evolutionary Computation Conference Companion (GECCO '23)*, Lisbon, Portugal, 2023. ACM, ACM.

[15] Maciej Komosinski and Konrad Miazga. Gom-based compatible substitutions optimization for variable-length representation gray-box problems. In *Genetic and Evolutionary Computation Conference (GECCO '25 Companion)*. ACM, ACM, 2025.

[16] Maciej Komosinski and Jan Polak. Evolving free-form stick ski jumpers and their neural control systems. In *Proceedings of the National Conference on Evolutionary Computation and Global Optimization*, pages 103–110, Poland, 2009.

[17] Maciej Komosinski and Szymon Ulatowski. Framsticks website (last accessed 1 jul 2026), 1996. http://www.framsticks.com/.

[18] Maciej Komosinski and Szymon Ulatowski. Gecco 2026: Automated design competition website (last accessed 1 jul 2026), 1996. http://www.framsticks.com/gecco-competition.

[19] Maciej Komosinski and Szymon Ulatowski. *Framsticks: towards a simulation of a nature-like world, creatures and evolution*. Advances in Artificial Life. Lecture Notes in Artificial Intelligence 1674. Springer-Verlag, 1999.

[20] Grzegorz Latosiński. Development and comparison of genetic representations for evolving 3d structures. Master's thesis, Poznan University of Technology, Institute of Computing Science, 2018.

[21] Risto Miikkulainen. Neuroevolution insights into biological neural computation. *Science*, 387(6735):eadp7478, 2025.

[22] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015.

[23] Jed Muff, Keiichi Ito, Elijah H. W. Ang, Karine Miras, and A. E. Eiben. Unconventional hexacopters via evolution and learning: Performance gains and new insights, 2026.

[24] Kouhei Nishida and Youhei Akimoto. Psa-cma-es: Cma-es with population size adaptation. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '18, page 865–872, New York, NY, USA, 2018. Association for Computing Machinery.

[25] Masahiro Nomura, Youhei Akimoto, and Isao Ono. Cma-es with learning rate adaptation. *ACM Trans. Evol. Learn. Optim.*, 5(1), March 2025.

[26] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning, 2017.

[27] Xiang Shu, Yiyi Zhu, Renji Zhang, Xiang Xia, Bingdong Li, and Hong Qian. Automated design competition technical report: Cascaded structure and parameter optimization based on prior knowledge. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, GECCO '24 Companion, page 3–4, New York, NY, USA, 2024. Association for Computing Machinery.

[28] K. O. Stanley and R. Miikkulainen. Competitive coevolution through evolutionary complexification. *Journal of Artificial Intelligence Research*, 21:63–100, February 2004.

[29] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.

[30] Marco Virgolin, Tanja Alderliesten, Cees Witteveen, and Peter A. N. Bosman. Scalable genetic programming by gene-pool optimal mixing and input-space entropy-based building-block learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '17, page 1041–1048, New York, NY, USA, 2017. Association for Computing Machinery.

[31] Enrico Zardini, Davide Zappetti, Davide Zambrano, Giovanni Iacca, and Dario Floreano. Seeking quality diversity in evolutionary co-design of morphology and control of soft tensegrity modular robots. *CoRR*, abs/2104.12175, 2021.

[32] Agata Żywot, Zuzanna Gawrysiak, Bartosz Stachowiak, Daniel Jankowski, and Kacper Dobek. Adaptmut+diversity competition submission repository (last accessed 1 jul 2026), 2024. https://github.com/kapiblue/frams-gecco-2024.


no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnone
DynMutF0pmut08wHist_scorefnneg
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0675_pops20_pxov0675_restsoftperturbbest_rest77_tour12_xmut1
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
AdaptMutF0pmut08pop30initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
DynMutF0pmut0675pop20pxov0675restart_patience77tournament12wHist_scorefnneg
DynMutF0pmut0675pop20pxov0675restart_patience77tournament12wHist_scorefnnegwHist_decay0965
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selqualityrestart_methodsoftperturbbestrestart_patience100tournament50
AdaptMutF0pmut0675pop20initialgenotyperandomadded_indinitialpxov0675restart_methodsoftperturbbestrestart_patience77tournament12
DynMutF0pmut08wHist_scorefnratiofifthrule
DynMutF0pmut08wHist_scorefnnegxmut_enabled0
DynMutF0pmut08wHist_scorefnratio
AdaptMutF0pmut08lbda25added_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
DynMutF0pmut08wHist_scorefnnegwHist_decay093
AdaptMutF0initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10pxov0
AdaptMutF0pmut08added_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_scorefnneg
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_ESalgoindstorexov_mutschemachoice
DynMutF0pmut08wHist_scorefnnegwHist_decay095
AdaptMutF0pmut0675pop20initialgenotyperandomadded_indrandompxov0675restart_methodsoftperturbbestrestart_patience77tournament12xmut_enabled1
AdaptMutF0pmut08initialgenotyperandomadded_indrandom
AdaptMutF0pmut08pop100initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_ESalgoindstorexov_mutschemarand
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0705_pops121_pxov055_restsoftperturbbest_rest80_tour4_xmut1
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr19_nisl6_pmut085_pops465_pxov03_tour7
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut065_pops24_pxov0645_restsoftperturbbest_rest81_tour9_xmut1
AdaptMutF0
no_det_DynMutF0pmut08wHist_scorefnneg
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr24_nisl6_pmut098_pops495_pxov0115_tour3
DynMutF0pmut08wHist_scorefnratiov2
DynMutF0pmut08wHist_scorefnnegconservative
AdaptMutF0pmut08fix_invalidmutate
DynMutF0pmut08restart_methodsoftperturbbestallwHist_scorefnneg
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYflibclasswHistwHist_scorefnneg
DynMutF0pmut08wHist_scorefnpos
DynMutF0pmut08Actual_noignore_fitness
convection_AdaptMutF0pop500
DynMutF0pmut08wHist_scorefnconst
no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesignoreinfeasible
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0684999999999999_pops25_pxov0410000000000003_tour12_xmut1
AdaptMutF0pmut08pop25
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0775_pops116_pxov0355_resthard_rest25_tour13_xmut1
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestallrestart_patience10
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut073_pops6_pxov042_restsoftperturbbest_rest75_tour6_xmut1
AdaptMutF0pmut08selMethodrouletteinitialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0995_pops54_pxov0685_restsoftperturbbest_rest30_tour15_xmut1
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr21_nisl6_pmut071_pops465_pxov0275_tour7
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetarestart_methodsoftperturbbestrestart_patience100tournament50
AdaptMutF0pmut08pop250initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
AdaptMutF0pmut08initialgenotyperandomadded_indrandomrestart_methodsoftperturbbestrestart_patience1000000flibclasswHistwHist_scorefnneg
no_det_convection_AdaptMutF0pop500
NEAT_speciationF0pop75initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDY
AdaptMutF0pmut08added_indrandom
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr23_nisl5_pmut074_pops465_pxov0295_tour7
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut049_pops11_pxov0665_restsoftperturbbest_rest81_tour4_xmut1
no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesignoreinfeasiblecon straints
convection_eaSimpleF1
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_initf0XX_pmut072_pops59_pxov0395_resthard_rest27_tour7_xmut1
convection_AdaptMutF0
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf0XXneurons_pmut078_pops58_pxov0745_restsoftperturbbest_rest2_tour9_xmut1
AdaptMutF0initialgenotype0_f0_basic2
convection_eaSimpleF0
convection_eaSimpleF1pmut08
AdaptMutF0initialgenotype0_f0_neurons
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr18_nisl3_pmut0570000000000001_pops410_pxov0415000000000004_tour7
AdaptMutF0pmut08
no_det_DynMutF1pmut08wHist_scorefnneg
convection_AdaptMutF0xmut_enabled0
NEAT_speciationF0popsize100nislands10dissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience15initialgenotyperandom
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut06_pops21_pxov0665_restsoftperturbbest_rest96_tour13_xmut1
AdaptMutF0initialgenotyperandom
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_pmut07_pops135_pxov0015_tour5_xmut1
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_initf0XX_pmut076_pops85_pxov0935_restsoftperturbbest_rest38_tour14_xmut1
no_det_convection_eaSimpleF1pop100
convection_eaSimpleF0initialgenotyperandomadded_indrandom
convection_AdaptMutF1pop100xmut_enabled0
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf1_initf1random_pmut071_pops56_pxov058_restsoftperturbbest_rest77_tour5_xmut1
convection_AdaptMutF0pmut08
convection_eaSimpleF0pmut08
AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_decay10
convection_AdaptMutF1pmut08xmut_enabled0
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr30_nisl7_pmut0855_pops425_pxov027_tour3
no_det_DynMutF1pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnone
convection_eaSimpleF1pop200
convection_eaSimpleF1pmut07
AdaptMutF0pmut08pop100
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut066_pops45_pxov0055_restsoftperturbbest_rest19_tour8_xmut1
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf1_initf1random_pmut065_pops106_pxov063_restsoftperturbbest_rest69_tour12_xmut1
convection_AdaptMutF1pmut08
m_evalfn3_NEAT_speciation_algoNEATspeciation_dissGENELEVENSHTEIN_genf0_initf1random_nisl3_pmut0755_pops7_pxov003_tour11
convection_AdaptMutF1
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetatournament50
convection_AdaptMutF0initialgenotyperandomadded_indrandom
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommeterarestart_methodsoftperturbbestrestart_patience100tournament25
no_det_AdaptMutF0pmut08
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0679999999999999_pops31_pxov038_restsoftperturbbest_rest2_tour7_xmut1
MAPElitesF0pmut0675pxov0675popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommeterarestart_methodsoftperturbbestrestart_patience100tournament50
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr24_nisl5_pmut0735_pops460_pxov03_tour7
convection_eaSimpleF1pop100
NEAT_speciationF0pop100
no_det_AdaptMutF0
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0565000000000001_pops14_pxov0015_restsoftperturbbest_rest2_tour9_xmut1
NEAT_speciationF0dissimPHENESTRUCTOPTIM
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XXneurons_islabestToWorst_migr22_nisl20_pmut0655_pops440_pxov0475000000000003_tour5
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr44_nisl7_pmut0965_pops465_pxov002_tour2
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr23_nisl6_pmut0975_pops495_pxov011_tour3
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr25_nisl4_pmut0765_pops500_pxov089_tour12
m_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0595_pops84_pxov039_restsoftperturbbest_rest3_tour11_xmut1
NEAT_speciationF0pop200nislands15
DynMutF1pmut08wHist_scorefnneg
convection_eaSimpleF1pmut08pop100
NEAT_speciationF0dissimPHENESTRUCTGREEDY
m_evalfn3_eaSimple_algoeaSimple_genf0_initf1random_pmut07_pops158_pxov0570000000000001_tour2
AdaptMutF1initialgenotyperandom
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf1_pmut078_pops116_pxov03500000000000003_restsoftperturbbest_rest11_tour12_xmut1
AdaptMutF1initialgenotypeXX
m_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf1random_lbda245_pmut066_pops120_pxov0339999999999999997_tour8
m_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0715_pops124_pxov059_restsoftperturbbest_rest81_tour2_xmut1
m_evalfn3_eaSimple_algoeaSimple_genf0_pmut0615_pops33_pxov0015_tour3
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf1_initf1XX_islabestToWorst_migr24_nisl5_pmut08_pops410_pxov0595_tour6
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr20_nisl29_pmut0725_pops488_pxov038_tour3
NEAT_speciationF1dissimPHENESTRUCTGREEDY
eaSimpleF0
NEAT_speciationF0nislands5
NEAT_speciationF0dissimFITNESS
m_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf0XX_lbda500_pmut0635_pops500_pxov0365_tour6
NEAT_speciationF0pop100nislands5
m_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf1random_lbda230_pmut069_pops115_pxov0310000000000005_tour9
eaSimpleF0initialgenotyperandom
DynMutF0pmut08wHist_scorefnnegwHist_decay1
m_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf1random_lbda240_pmut0785_pops25_pxov0214999999999999997_tour2
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr35_nisl60_pmut0695_pops446_pxov0405_tour6
AdaptMutF1
NEAT_speciationF1nislands5
eaOnePlusLambdaLambdaF0pop1
m_evalfn3_AdaptMut_algoAdaptMut_genf0_pmut0535_pops435_pxov048_tour4_xmut1
AdaptMutF1initialgenotypeXXGGpartTSN
NEAT_speciationF1dissimPHENESTRUCTOPTIM
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr1_nisl14_pmut10_pops495_pxov03_tour7
eaMuPlusLambdaF0pmut08lbda100
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr41_nisl81_pmut069_pops491_pxov044_tour9
eaSimpleF0initialgenotype0_f0_basic2
eaMuCommaLambdaF0pmut08lbda100
m_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf0_initf1random_pmut0235000000000001_pops1_pxov0365
m_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf1random_lbda213_pmut0785_pops33_pxov0214999999999999997_tour6
eaSimpleF0restart_methodsoftperturbbestrestart_patience10
NEAT_speciationF1dissimFITNESS
eaSimpleF0initialgenotype0_f0_neurons
m_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf1_pmut062_pops1_pxov0035
m_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr21_nisl79_pmut071_pops488_pxov075_tour9
eaSimpleF0pmut08
m_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_lbda500_pmut0809999999999999_pops500_pxov0190000000000006_tour5
AdaptMutF1pmut08
eaMuPlusLambdaF0pmut08lbda350
m_evalfn3_eaSimple_algoeaSimple_genf0_pmut0325_pops111_pxov004_tour9
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetarestart_methodsoftperturbbestrestart_patience100tournament50
eaMuCommaLambdaF0pmut08lbda350
m_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf0XX_lbda500_pmut084_pops365_pxov0160000000000003_tour10
eaOnePlusLambdaLambdaF0pop1initialgenotype0
eaSimpleF1initialgenotypeXX
m_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr28_nisl51_pmut076_pops109_pxov0350000000000003_tour10
m_evalfn3_eaSimple_algoeaSimple_genf1_initf1random_pmut0635_pops85_pxov006_tour11
eaSimpleF1pmut08
eaSimpleF1initialgenotyperandom
NEAT_speciationF1pop100nislands5
eaSimpleF1
MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selcuriosityrestart_methodsoftperturbbestrestart_patience100tournament50
no_det_eaSimpleF1
NEAT_speciationF1dissimGENELEVENSHTEIN
m_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_initf0XXneurons_islabestToWorst_migr48_nisl99_pmut076_pops165_pxov0025_tour5
eaSimpleF1initialgenotypeXXGGpartTSN
AdaptMutF0pmut08selMethodbestinitialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
eaMuCommaLambdaF1pmut08lbda350
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr7_nisl17_pmut01_pops407_pxov0830000000000001_tour2
eaMuPlusLambdaF1pmut08lbda100
m_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr36_nisl2_pmut079_pops3_pxov007_tour3
eaMuCommaLambdaF1pmut08lbda100
eaMuPlusLambdaF1pmut08lbda350
m_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf1_lbda416_pmut0775_pops341_pxov0224999999999999998_tour19
m_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf1random_islabestToWorst_migr28_nisl59_pmut0515_pops86_pxov078_tour15
NEAT_speciationF1dissimPHENEDENSITYFREQ
NEAT_speciationF1dissimPHENEDENSITYCOUNT
NEAT_speciationF1dissimPHENEDESCRIPTORS
m_evalfn3_eaSimple_algoeaSimple_genf0_initf1random_pmut073_pops1_pxov0025_tour8
m_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf1_pmut067_pops1_pxov0275
0
100
200
300
400
500
600
700

B List of Runs

B.1 List of Runs (evalfn3)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
1.	118.56489	**361.60925**	318.37850	645.18800	(0/20)	no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnone
2.	110.28801	345.41403	**337.74450**	580.82600	(0/30)	DynMutF0pmut08wHist_scorefnneg
3.	128.03199	342.58060	326.74850	608.65300	(0/20)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0675_pops20_pxov0675_restsoftperturbbest_rest77_tour12_xmut1
4.	111.97438	331.45400	304.09750	550.99900	(0/20)	AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
5.	102.77578	330.52145	320.04050	548.32400	(0/20)	AdaptMutF0pmut08pop30initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
6.	104.81810	329.57775	307.57950	606.28900	(0/20)	DynMutF0pmut0675pop20pxov0675restart_patience77tournament12wHist_scorefnneg
7.	123.45178	328.82731	318.28450	658.92900	(0/20)	DynMutF0pmut0675pop20pxov0675restart_patience77tournament12wHist_scorefnnegwHist_decay0965

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
8.	118.59155	326.90425	306.65950	679.97300	(0/20)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selqualityrestart_methodsoftperturbbestrestart_patience100tournament50
9.	122.42487	324.94940	299.01250	626.36200	(0/20)	AdaptMutF0pmut0675pop20initialgenotyperandomadded_indinitialpxov0675restart_methodsoftperturbbestrestart_patience77tournament12
10.	134.25382	319.74365	288.59050	585.24100	(0/20)	DynMutF0pmut08wHist_scorefnratiofifthrule
11.	97.16209	314.85470	291.77600	557.78800	(0/20)	DynMutF0pmut08wHist_scorefnnegxmut_enabled0
12.	88.82923	311.49910	317.95350	456.20800	(0/20)	DynMutF0pmut08wHist_scorefnratio
13.	84.49988	309.37380	302.49750	468.78500	(0/20)	AdaptMutF0pmut08lbda25added_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
14.	106.85136	306.89310	265.82100	567.40000	(0/20)	DynMutF0pmut08wHist_scorefnnegwHist_decay093
15.	97.59817	305.53770	259.04250	586.95500	(0/20)	AdaptMutF0initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10pxov0
16.	68.80471	304.58320	291.93150	499.29900	(0/20)	DynMutF0pmut08

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
17.	98.29305	302.81513	301.69750	475.45600	(0/20)	AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_ESalgoindstorexov_mutschemachoice
18.	108.49403	302.76495	270.41350	516.86600	(0/20)	DynMutF0pmut08wHist_scorefnnegwHist_decay095
19.	97.48249	294.72713	279.29700	455.23300	(0/20)	AdaptMutF0pmut0675pop20initialgenotyperandomadded_indrandompxov0675restart_methodsoftperturbbestrestart_patience77tournament12xmut_enabled1
20.	103.15403	293.27265	266.98850	518.02800	(0/20)	AdaptMutF0pmut08initialgenotyperandomadded_indrandom
21.	106.87032	290.43345	325.88800	521.60900	(0/20)	AdaptMutF0pmut08pop100initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
22.	102.78720	289.99870	240.26400	558.26000	(0/20)	AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10flibclasswHistwHist_ESalgoindstorexov_mutschemarand
23.	103.29228	289.90270	258.58600	535.15500	(0/20)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0705_pops121_pxov055_restsoftperturbbest_rest80_tour4_xmut1

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
24.	120.81813	289.86785	275.52600	529.32700	(0/20)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr19_nisl6_pmut085_pops465_pxov03_tour7
25.	114.94601	286.66845	267.44150	524.85500	(0/20)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut065_pops24_pxov0645_restsoftperturbbest_rest81_tour9_xmut1
26.	79.87178	285.36440	272.86650	528.28200	(0/20)	AdaptMutF0
27.	77.16551	284.73340	263.99200	422.69100	(0/20)	no_det_DynMutF0pmut08wHist_scorefnneg
28.	125.82745	283.17560	272.94300	595.53200	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr24_nisl6_pmut098_pops495_pxov0115_tour3
29.	133.68427	282.76680	242.10850	560.34300	(0/10)	DynMutF0pmut08wHist_scorefnratiov2
30.	83.35859	281.50755	267.16100	485.09700	(0/20)	DynMutF0pmut08wHist_scorefnnegconservative
31.	82.81854	279.92820	288.10200	459.31700	(0/20)	AdaptMutF0pmut08fix_invalidmutate
32.	86.90849	277.86785	282.18550	505.65900	(0/20)	DynMutF0pmut08restart_methodsoftperturbbestallwHist_scorefnneg
33.	60.18809	277.33560	274.57800	360.78400	(0/10)	DynMutF0wHist_scorefnneg
34.	83.49951	275.89170	253.00300	447.68000	(0/20)	DynMutF0pmut08wHist_scorefnpos

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
35.	122.77318	273.07177	242.94950	634.97300	(0/20)	MAPElitesF0pop20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selqualityrestart_methodnonetournament50
36.	77.02442	272.55010	265.08100	413.54700	(0/20)	DynMutF0pmut08Actual_noignore_fitness
37.	83.55977	270.61985	287.27300	417.56500	(0/20)	convection_AdaptMutF0pop500
38.	98.24565	269.85420	252.74300	567.68100	(0/30)	DynMutF0pmut08wHist_scorefnconst
39.	81.44586	269.48820	236.52300	479.96700	(0/20)	no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesignoreinfeasible
40.	118.67421	267.96500	236.49000	**684.94600**	(0/20)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut06849999999999999_pops25_pxov0410000000000003_tour12_xmut1
41.	99.18738	266.63089	249.66900	486.33400	(0/20)	AdaptMutF0pmut08pop25
42.	58.36792	264.93970	234.42800	367.80700	(0/10)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0775_pops116_pxov0355_resthard_rest25_tour13_xmut1
43.	66.77494	261.18310	258.63550	389.02800	(0/20)	AdaptMutF0pmut08initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestallrestart_patience10
44.	51.76372	259.74575	262.11900	356.24500	(0/20)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut073_pops6_pxov042_restsoftperturbbest_rest75_tour6_xmut1

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
45.	91.60685	257.50975	248.16950	565.93500	(0/20)	AdaptMutF0pmut08selMethodrouletteinitialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
46.	102.15915	256.33370	252.95950	505.72300	(0/10)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0995_pops54_pxov0685_restsoftperturbbest_rest30_tour15_xmut1
47.	45.08143	256.26170	251.66100	344.25600	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr21_nisl6_pmut071_pops465_pxov0275_tour7
48.	84.10663	254.50927	235.60850	507.45900	(0/40)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetarestart_methodsoftperturbbestrestart_patience100tournament50
49.	89.02414	253.70295	247.94150	466.64000	(0/40)	AdaptMutF0pmut08pop250initialgenotyperandomaddedd_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
50.	86.16816	253.40180	249.92550	519.37400	(0/20)	DynMutF0pmut08restart_methodnonewHist_scorefnneg
51.	76.19516	252.28110	238.67850	400.73700	(0/20)	no_det_convection_AdaptMutF0pop500
52.	60.06200	250.20824	265.23500	326.23300	(0/20)	NEAT_speciationF0pop75initialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDY

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
53.	102.88860	249.01476	243.49050	509.24100	(0/20)	*AdaptMutF0pmut08added_indrandom*
54.	122.67286	248.97780	211.05800	580.27500	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr23_nisl5_pmut074_pops465_pxov0295_tour7
55.	87.02664	246.97250	224.06150	487.45100	(0/20)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut049_pops11_pxov0665_restsoftperturbbest_rest81_tour4_xmut1
56.	89.67190	246.06993	239.51300	434.40500	(0/30)	no_det_DynMutF0pmut08wHist_scorefnnegwHist_ignorefitnesscmaesignoreinfeasibleconstraints
57.	75.62944	245.00700	258.95100	379.95700	(0/20)	convection_eaSimpleF1
58.	39.66782	244.64800	250.33750	301.13400	(0/20)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_initf0XX_pmut072_pops59_pxov0395_resthard_rest27_tour7_xmut1
59.	68.06847	243.16535	233.65450	381.50200	(0/20)	convection_AdaptMutF0
60.	125.57638	243.00344	219.46000	518.69400	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf0XXneurons_pmut078_pops58_pxov0745_restsoftperturbbest_rest2_tour9_xmut1
61.	83.07671	240.69130	237.80850	469.03600	(0/20)	AdaptMutF0initialgenotype0_f0_basic2
62.	76.20256	237.87615	233.75750	375.80300	(0/20)	convection_eaSimpleF0
63.	76.41129	237.25845	237.04800	408.54700	(0/20)	convection_eaSimpleF1pmut08

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
64.	94.69579	237.20980	263.34450	430.82200	(0/20)	AdaptMutF0initialgenotype 0_f0_neurons
65.	110.34577	235.89450	210.62450	558.28300	(0/10)	rn_evalfn3_convection_Adap tMut_algoconvectionAdapt Mut_genf0_initf0XX_islabest ToWorst_migr18_nisl3_pmut 05700000000000001_pops41 0_pxov041500000000000004 _tour7
66.	110.93452	234.14575	235.27100	543.96200	(0/20)	AdaptMutF0pmut08
67.	92.56128	234.04941	218.34800	438.23900	(0/10)	no_det_DynMutF1pmut08w Hist_scorefnneg
68.	66.50227	233.40955	218.69150	436.75800	(0/20)	convection_AdaptMutF0xm ut_enabled0
69.	68.40235	232.52396	217.55700	447.50900	(2/28)	NEAT_speciationF0popsize 100nislands10dissimPHENE STRUCTGREEDYrestart_ methodsoftperturbbestresta rt_patience15initialgenotype random
70.	76.21921	231.36780	214.71050	382.48100	(0/10)	rn_evalfn3_AdaptMut_addei nitial_algoAdaptMut_genf0_ initf1random_pmut06_pops2 1_pxov0665_restsoftperturb best_rest96_tour13_xmut1
71.	77.69570	230.91239	211.83950	350.89000	(0/10)	AdaptMutF0initialgenotype random
72.	88.12868	229.04553	240.24650	337.70500	(0/10)	rn_evalfn3_AdaptMut_addei nitial_algoAdaptMut_genf0_ pmut07_pops135_pxov0015_ tour5_xmut1
73.	59.58351	228.82040	211.21350	359.47600	(0/10)	rn_evalfn3_AdaptMut_adder andom_algoAdaptMut_genf 0_initf0XX_pmut076_pops85 _pxov0935_restsoftperturbbe st_rest38_tour14_xmut1

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
74.	99.34002	228.31045	222.26500	444.28900	(0/20)	no_det_convection_eaSimpleF1pop100
75.	56.32415	228.25305	222.77950	322.31200	(0/20)	convection_eaSimpleF0initialgenotyperandomadded_indrandom
76.	76.60084	228.22349	219.10850	437.91500	(0/20)	convection_AdaptMutF1pop100xmut_enabled0
77.	65.45451	228.12980	229.77300	314.20400	(0/10)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf1_initf1random_pmut071_pops56_pxov058_restsoftperturbbest_rest77_tour5_xmut1
78.	68.59033	227.03850	218.13300	492.87700	(0/60)	convection_AdaptMutF0pmut08
79.	61.89369	227.00630	222.91400	383.42600	(0/20)	convection_eaSimpleF0pmut08
80.	96.41741	227.00315	201.45100	522.81500	(0/20)	DynMutF0wHist_decay10
81.	72.61134	226.32945	206.06100	363.00300	(0/20)	convection_AdaptMutF1pmut08xmut_enabled0
82.	87.30806	226.05770	189.55650	447.52000	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr30_nisl7_pmut0855_pops425_pxov027_tour3
83.	101.63528	223.86099	199.36350	505.74600	(0/20)	no_det_DynMutF1pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnone
84.	73.85333	223.47885	215.32750	350.03900	(0/20)	convection_eaSimpleF1pop200
85.	68.95538	222.61520	192.88100	323.21800	(0/20)	convection_eaSimpleF1pmut07
86.	82.98468	222.51854	206.76750	415.99400	(0/20)	AdaptMutF0pmut08pop100

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
87.	60.62238	221.44330	202.37500	329.46400	(0/10)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut066_pops45_pxov0055_restsoftperturbbest_rest19_tour8_xmut1
88.	96.47617	218.16910	183.35400	373.00300	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf1_initf1random_pmut065_pops106_pxov063_restsoftperturbbest_rest69_tour12_xmut1
89.	65.07585	218.12545	224.04450	351.91400	(0/20)	convection_AdaptMutF1pmut08
90.	61.55864	216.86779	228.40650	296.21800	(0/10)	rn_evalfn3_NEAT_speciation_algoNEATspeciation_dissGENELEVENSHTEIN_genf0_initf1random_nisl3_pmut0755_pops7_pxov003_tour11
91.	61.87346	213.96295	206.66400	365.82300	(0/20)	convection_AdaptMutF1
92.	73.96963	213.25335	194.79800	338.67200	(0/20)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetatournament50
93.	56.00542	212.95135	202.77800	356.09700	(0/20)	convection_AdaptMutF0initialgenotyperandomadded_indrandom
94.	69.87975	211.34505	196.68300	395.87400	(0/20)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetarestart_methodsoftperturbbestrestart_patience100tournament25
95.	80.11475	209.76748	224.81550	415.27300	(0/20)	no_det_AdaptMutF0pmut08

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
96.	66.51568	209.73650	185.65150	374.40500	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0679999999999999_pops31_pxov038_restsoftperturbbest_rest2_tour7_xmut1
97.	73.25258	208.26150	192.66050	412.64400	(0/20)	MAPElitesF0pmut0675pxov0675popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandommetarestart_methodsoftperturbbestrestart_patience100tournament50
98.	55.63186	207.42730	193.13900	333.87500	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr24_nisl5_pmut0735_pops460_pxov03_tour7
99.	111.40678	207.12883	193.75100	532.74400	(0/20)	convection_eaSimpleF1pop100
100.	93.26182	206.14497	168.68200	410.86000	(0/20)	NEAT_speciationF0pop100
101.	105.09188	205.27518	192.79650	554.37200	(0/20)	no_det_AdaptMutF0
102.	43.17245	203.42930	199.95300	288.89000	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut05650000000000001_pops14_pxov0015_restsoftperturbbest_rest2_tour9_xmut1
103.	73.01679	199.81714	187.91600	376.39000	(0/39)	NEAT_speciationF0dissimPHENESTRUCTOPTIM
104.	50.34269	198.82950	196.75200	289.71000	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XXneurons_islabestToWorst_migr22_nisl20_pmut0655_pops440_pxov047500000000000003_tour5

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
105.	80.20317	198.37750	174.87850	379.49800	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr44_nisl7_pmut0965_pops465_pxov002_tour2
106.	68.90135	198.14890	185.83250	310.82700	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr23_nisl6_pmut0975_pops495_pxov011_tour3
107.	50.96380	196.63790	187.82050	285.83900	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr25_nisl4_pmut0765_pops500_pxov089_tour12
108.	73.73456	193.74000	169.70600	378.94300	(0/10)	rn_evalfn3_AdaptMut_adderandom_algoAdaptMut_genf0_pmut0595_pops84_pxov039_restsoftperturbbest_rest3_tour11_xmut1
109.	40.83007	190.73325	182.97050	309.40200	(0/20)	NEAT_speciationF0pop200nislands15
110.	102.48241	190.27813	131.56450	368.17100	(0/10)	DynMutF1pmut08wHist_scorefnneg
111.	73.95906	189.64229	177.21600	390.06700	(0/20)	convection_eaSimpleF1pmut08pop100
112.	57.04898	189.49424	180.27850	303.23000	(0/20)	NEAT_speciationF0dissimPHENESTRUCTGREEDY
113.	103.76094	188.69956	189.65600	474.34500	(0/20)	rn_evalfn3_eaSimple_algoeaSimple_genf0_initf1random_pmut07_pops158_pxov05700000000000001_tour2
114.	77.59363	183.63752	173.61600	361.28300	(0/10)	AdaptMutF1initialgenotyperandom

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
115.	72.23801	182.61131	157.29050	353.59800	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf1_pmut078_pops116_pxov0350000000000003_restsoftperturbbest_rest11_tour12_xmut1
116.	86.04574	181.19126	150.70550	349.12000	(0/20)	AdaptMutF1initialgenotypeXX
117.	65.36974	177.93512	170.24400	312.38400	(0/20)	rn_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf1random_lbda245_pmut066_pops120_pxov033999999999997_tour8
118.	51.59143	176.70240	154.11850	269.15600	(0/10)	rn_evalfn3_AdaptMut_addeinitial_algoAdaptMut_genf0_initf1random_pmut0715_pops124_pxov059_restsoftperturbbest_rest81_tour2_xmut1
119.	42.74749	176.14970	186.08450	245.84500	(0/10)	rn_evalfn3_eaSimple_algoeaSimple_genf0_pmut0615_pops33_pxov0015_tour3
120.	75.91908	175.99690	151.50400	331.23400	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf1_initf1XX_islabestToWorst_migr24_nisl5_pmut08_pops410_pxov0595_tour6
121.	54.50076	174.87040	154.07750	297.60500	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr20_nisl29_pmut0725_pops488_pxov038_tour3
122.	62.84837	172.29113	157.24650	318.19800	(3/20)	NEAT_speciationF1dissimPHENESTRUCTGREEDY
123.	97.42668	171.55590	146.49150	406.93200	(0/20)	eaSimpleF0
124.	61.32771	169.08129	159.87450	274.47900	(0/20)	NEAT_speciationF0nislands5

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
125.	74.73547	167.46402	155.23850	357.24000	(0/20)	NEAT_speciationF0dissimFITNESS
126.	92.60794	166.60894	144.80900	402.12800	(0/20)	rn_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf0XX_lbda500_pmut0635_pops500_pxov0365_tour6
127.	60.30816	163.28717	166.47900	334.73300	(0/20)	NEAT_speciationF0pop100nislands5
128.	74.96505	161.70664	168.61600	297.67400	(0/10)	rn_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf1random_lbda230_pmut069_pops115_pxov031000000000000005_tour9
129.	56.87945	161.56651	138.37400	250.22900	(0/10)	eaSimpleF0initialgenotyperandom
130.	61.49788	160.86270	164.83750	287.16600	(0/10)	DynMutF0pmut08wHist_scorefnnegwHist_decay1
131.	36.26507	159.07465	153.75300	233.91200	(0/10)	rn_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf1random_lbda240_pmut0785_pops25_pxov021499999999999997_tour2
132.	38.21639	158.67507	158.06450	237.81800	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr35_nisl60_pmut0695_pops446_pxov0405_tour6
133.	102.29949	158.55645	140.20150	441.26800	(0/20)	AdaptMutF1
134.	63.03672	158.54638	154.61500	315.77500	(1/20)	NEAT_speciationF1nislands5
135.	70.11401	158.53893	138.54300	336.39300	(0/40)	eaOnePlusLambdaLambdaF0pop1

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
136.	46.01961	153.61436	144.87000	253.04200	(0/10)	rn_evalfn3_AdaptMut_algoAdaptMut_genf0_pmut0535_pops435_pxov048_tour4_xmut1
137.	97.49192	147.39023	123.26300	383.96800	(0/20)	AdaptMutF1initialgenotypeXXGGpartTSN
138.	99.68314	146.96174	115.31300	440.29700	(4/39)	NEAT_speciationF1dissimPHENESTRUCTOPTIM
139.	67.68371	145.57736	131.59850	260.21800	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf0XX_islabestToWorst_migr1_nisl14_pmut10_pops495_pxov03_tour7
140.	45.91179	145.26954	144.85750	230.89800	(0/20)	eaMuPlusLambdaF0pmut08lbda100
141.	28.74714	144.45570	134.20250	206.65400	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr41_nisl81_pmut069_pops491_pxov044_tour9
142.	42.67774	142.36872	132.55850	262.88300	(0/20)	eaSimpleF0initialgenotype0_f0_basic2
143.	46.16849	140.86175	122.53750	224.99800	(0/20)	eaMuCommaLambdaF0pmut08lbda100
144.	58.64210	140.14862	122.09950	250.84900	(0/10)	rn_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf0_initf1random_pmut0235000000000001_pops1_pxov0365
145.	60.65619	139.72842	125.46100	292.21700	(0/10)	rn_evalfn3_eaMuCommaLambda_algoeaMuCommaLambda_genf0_initf1random_lbda213_pmut0785_pops33_pxov021499999999999997_tour6

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
146.	81.45552	134.37733	127.85850	433.85100	(0/20)	eaSimpleF0restart_methodsoftperturbbestrestart_patience10
147.	97.09776	133.65598	107.94950	335.57800	(0/20)	NEAT_speciationF1dissimFITNESS
148.	71.88942	130.98951	126.90950	342.90200	(0/20)	eaSimpleF0initialgenotype0_f0_neurons
149.	45.19881	130.69818	135.26300	215.82000	(0/10)	rn_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf1_pmut062_pops1_pxov0035
150.	27.65701	130.11578	120.58100	172.70400	(0/10)	rn_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr21_nisl79_pmut071_pops488_pxov075_tour9
151.	68.52382	129.94296	109.77900	332.84800	(0/20)	eaSimpleF0pmut08
152.	61.58722	129.18333	147.07950	221.11400	(0/10)	rn_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_lbda500_pmut0809999999999999_pops500_pxov019000000000000006_tour5
153.	71.52893	126.27572	95.65995	321.39800	(0/20)	AdaptMutF1pmut08
154.	66.95189	122.10886	89.82540	279.38800	(0/20)	eaMuPlusLambdaF0pmut08lbda350
155.	64.57591	121.83612	138.47600	197.28800	(0/10)	rn_evalfn3_eaSimple_algoeaSimple_genf0_pmut0325_pops111_pxov004_tour9
156.	57.18260	120.94354	108.61950	285.34300	(0/20)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selrandomrestart_methodsoftperturbbestrestart_patience100tournament50
157.	46.51430	120.22287	112.46950	224.62600	(0/20)	eaMuCommaLambdaF0pmut08lbda350

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
158.	50.13103	119.74849	104.49100	245.62800	(0/10)	rn_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf0_initf0XX_lbda500_pmut084_pops365_pxov0160000000000003_tour10
159.	47.05908	116.59629	119.23650	242.66000	(0/20)	eaOnePlusLambdaLambdaF0pop1initialgenotype0
160.	45.07542	100.32251	94.20590	204.39900	(0/20)	eaSimpleF1initialgenotypeXX
161.	21.97628	96.90388	102.58200	129.30900	(0/10)	rn_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr28_nisl51_pmut076_pops109_pxov0350000000000000003_tour10
162.	51.67827	94.73865	85.56765	217.86300	(0/10)	rn_evalfn3_eaSimple_algoeaSimple_genf1_initf1random_pmut0635_pops85_pxov006_tour11
163.	58.26179	94.44432	91.13255	251.47300	(0/20)	eaSimpleF1pmut08
164.	69.08071	89.77198	63.63310	263.54200	(0/10)	eaSimpleF1initialgenotyperandom
165.	53.30151	89.18115	73.74875	222.89700	(15/20)	NEAT_speciationF1pop100nislands5
166.	46.52943	87.73373	78.88120	186.71600	(0/20)	*eaSimpleF1*
167.	23.27419	81.54561	76.79185	144.88900	(0/20)	MAPElitesF0popsize20wHist_cacheActive1flibclasswHistwHist_ESalgononenovelty_selcuriosityrestart_methodsoftperturbbestrestart_patience100tournament50
168.	39.10576	81.30353	77.50035	176.81000	(0/20)	no_det_eaSimpleF1
169.	47.04598	74.61855	66.99510	194.15300	(0/20)	NEAT_speciationF1dissimGENELEVENSHTEIN

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
170.	12.97105	74.02647	72.55660	101.45500	(0/10)	rn_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_initf0XXneurons_islabestToWorst_migr48_nisl99_pmut076_pops165_pxov0025_tour5
171.	42.89914	73.12026	67.67320	204.55300	(0/20)	eaSimpleF1initialgenotypeXXGGpartTSN
172.	24.80531	71.30608	65.28875	153.88800	(0/20)	AdaptMutF0pmut08selMethodbestinitialgenotyperandomadded_indrandomdissimPHENESTRUCTGREEDYrestart_methodsoftperturbbestrestart_patience10
173.	29.02296	64.12219	61.02105	141.53600	(0/20)	eaMuCommaLambdaF1pmut08lbda350
174.	74.91810	60.89176	40.37395	283.34200	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_islabestToWorst_migr7_nisl17_pmut01_pops407_pxov083000000000000001_tour2
175.	56.08614	60.04540	57.69920	226.25700	(0/20)	eaMuPlusLambdaF1pmut08lbda100
176.	33.63212	57.10363	49.96515	151.52100	(0/10)	rn_evalfn3_convection_eaSimple_algoconvectioneaSimple_genf0_islabestToWorst_migr36_nisl2_pmut079_pops3_pxov007_tour3
177.	49.55113	53.89931	42.86660	224.64700	(0/20)	eaMuCommaLambdaF1pmut08lbda100
178.	33.83895	39.37166	19.75955	126.18800	(0/20)	eaMuPlusLambdaF1pmut08lbda350

Continued on next page

Table 5 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
179.	33.54458	38.91425	26.58150	113.21000	(0/10)	rn_evalfn3_eaMuPlusLambda_algoeaMuPlusLambda_genf1_lbda416_pmut0775_pops341_pxov0224999999999999998_tour19
180.	7.43011	34.52932	33.92020	44.64750	(0/10)	rn_evalfn3_convection_AdaptMut_algoconvectionAdaptMut_genf0_initf1random_islabestToWorst_migr28_nisl59_pmut0515_pops86_pxov078_tour15
181.	27.95409	26.19581	13.71005	86.35990	(20/20)	NEAT_speciationF1dissimPHENEDENSITYFREQ
182.	17.11060	13.11953	4.41728	63.76950	(20/20)	NEAT_speciationF1dissimPHENEDENSITYCOUNT
183.	25.35968	11.21698	3.47358	119.45800	(20/20)	NEAT_speciationF1dissimPHENEDESCRIPTORS
184.	6.91842	7.99027	6.75836	25.42220	(0/10)	rn_evalfn3_eaSimple_algoeaSimple_genf0_initf1random_pmut073_pops1_pxov0025_tour8
185.	0.00000	0.00000	0.00000	0.00000	(0/10)	rn_evalfn3_eaOnePlusLambdaLambda_algoeaOnePlusLambdaLambda_genf1_pmut067_pops1_pxov0275

Table 5: Leader board of each tested algorithm, on `evalfn3`.

B.2 List of Runs (evalfn4)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
1.	91.38086	**193.07575**	157.37500	**381.38700**	(0/20)	AdaptMutF0pmut0675pop20 evalfn4initialgenotyperandom added_indinitialpxov0675resta rt_methodsoftperturbbestrest art_patience77tournament12x mut_enabled1
2.	75.60346	173.58908	**169.24950**	378.77600	(0/20)	DynMutF0pmut08wHist_scor efnnegevalfn4
3.	53.51705	129.67909	126.52700	230.79800	(0/20)	AdaptMutF0pmut08evalfn4
4.	33.63458	127.38663	118.42150	212.94100	(0/20)	convection_AdaptMutF0pmut 08evalfn4
5.	59.40866	115.73766	96.86050	255.94200	(0/20)	*Adapt-MutF0pmut08added_indrandomevalfn4*
6.	79.69934	87.65928	56.86780	330.82600	(0/20)	no_det_DynMutF1pmut08wHi st_scorefnnegwHist_ignorefitn esscmaesnoneevalfn4
7.	39.43755	70.53197	68.78570	178.73500	(0/20)	eaSimpleF0evalfn4
8.	28.18626	45.68405	40.06590	130.71600	(0/20)	AdaptMutF1pmut08added_in drandomevalfn4
9.	34.42836	30.32477	19.63690	162.19600	(0/20)	*eaSimpleF1evalfn4*

Table 6: Leader board of each tested algorithm, on `evalfn4`.

B.3 List of Runs (evalfn5)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
1.	0.16630	**996.19785**	**996.23650**	996.35100	(0/20)	AdaptMutF0pmut08evalfn5
2.	0.11776	996.19115	996.16300	**996.40900**	(0/20)	*Adapt-MutF0pmut08added_indrandomevalfn5*
3.	0.19292	996.01015	996.10600	996.25800	(0/20)	DynMutF0pmut08wHist_scoref nnegevalfn5

Continued on next page

Table 7 (continued)

Nr.	Stddev	Mean	Median	Max	Overtime	Name
4.	0.22376	995.99905	996.07950	996.26600	(0/20)	AdaptMutF0pmut0675pop20evalfn5initialgenotyperandomadded_indinitialpxov0675restart_methodsoftperturbbestrestart_patience77tournament12xmut_enabled1
5.	0.32902	995.54830	995.50650	996.19100	(0/20)	no_det_DynMutF1pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnoneevalfn5
6.	0.21799	995.51480	995.54000	996.05100	(0/20)	AdaptMutF1pmut08added_indrandomevalfn5
7.	0.23873	995.50540	995.47050	996.00600	(0/20)	*eaSimpleF1evalfn5*
8.	0.29232	995.37800	995.39950	995.84400	(0/20)	eaSimpleF0evalfn5

Table 7: Leader board of each tested algorithm, on `evalfn5`.

B.4 List of Runs (evalfn6)

Nr.	Stddev	Mean	Median	Max	Name	
1.	0.00025	**0.00170**	**0.00163**	**0.00267**	(0/20)	no_det_DynMutF1pmut08wHist_scorefnnegwHist_ignorefitnesscmaesnoneevalfn6
2.	0.00012	0.00163	0.00160	0.00215	(0/20)	eaSimpleF0evalfn6
3.	0.00003	0.00162	0.00162	0.00165	(0/20)	*AdaptMutF0pmut08added_indrandomevalfn6*
4.	0.00007	0.00158	0.00158	0.00169	(0/20)	*eaSimpleF1evalfn6*

Table 8: Leader board of each tested algorithm, on `evalfn6`.